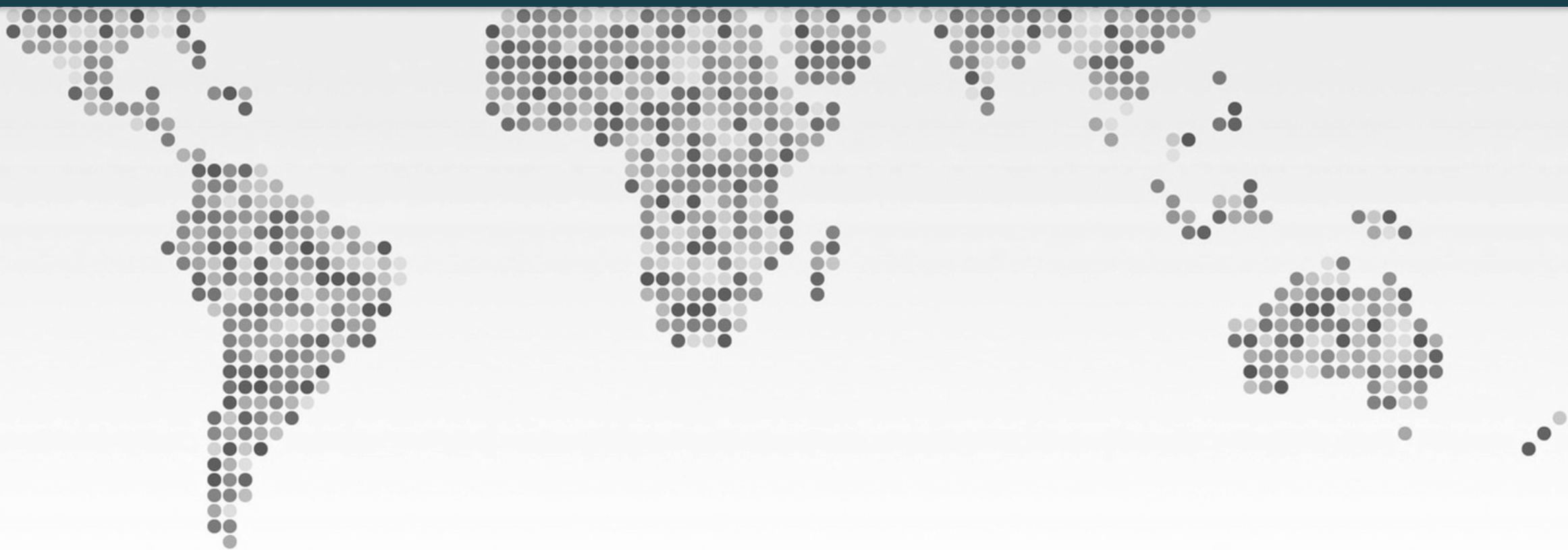


Software Engineering for Artificial Intelligence

Requirements Engineering for AI-based Systems



Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems

In software products, machine learning is typically used as a *component* in a larger system—often a very important component, but usually still just one among many components. Yet, most education on machine learning has an entirely *model-centric* view, focusing narrowly on the learning algorithms and building models, but rarely considering how the model would actually be used for a practical goal.

Adopting a *system-wide view* is important in many ways. We need to understand the goal of the system and how the model supports that goal but also introduces risks.

Key design decisions require an understanding of the full system and its environment, such as whether to act autonomously based on predictions or whether and when to check those predictions with non-ML code, with other models, or with humans in the loop. Such decisions matter substantially for how a system can cope with mistakes and has implications for usability, safety, and fairness.

To better understand the challenges of considering a system-wide view, let us discuss a realistic scenario.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: An Automated Transcription Start-Up



Sidney, a data scientist, has advanced speech recognition technology at a university, focusing on adapting it to specific domains by detecting technical terminology and jargon. The approach involved training neural networks on large datasets and using transfer learning on small, expert-annotated datasets.

Sidney showed the method's effectiveness in transcribing doctor-patient conversations, academic conference talks on poverty and inequality, and Ruby programming meetups. The research was published in top machine-learning conferences.

During the research, Sidney spoke with university researchers who frequently transcribe lengthy interviews and were frustrated with existing services. Most relied on human transcription services, which were costly (~\$1.50 per minute) and slow. Machine-generated transcription options, like YouTube's automated subtitles, existed but struggled with technical vocabulary. Conference organizers also sought live-captioning solutions for accessibility, but available options were either expensive (involving human transcribers) or produced low-quality results.

Based on the challenges encountered by the people she talked with, Sidney decides to try to commercialize a domain-specific speech-recognition technology by creating a start-up with some friends.

From Models to Systems: An Automated Transcription Start-Up

At this stage, Sidney quickly realizes that, even though the models trained in research are a great starting point, it will be a long path toward building a business with a stable product. To build a commercially viable product, there are lots of challenges:

1

In academia, Sidney's models outperformed the state-of-the-art models on accuracy measured on test data by a significant margin, but **audio files received from customers are often noisier than those used for benchmarking** in research.

2

In the research lab, it did not matter much how long it took to train the model or transcribe an audio file. Now customers **get impatient if their audio files are not transcribed within 15 minutes**. Worse, **live captioning** needs to be essentially **instantaneous**. Making model inference faster and better scalable to process many transcriptions suddenly becomes an important focus for the start-up. **Live captioning turns out to be unrealistic after all unless expensive specialized hardware is shipped to the conference venue to achieve latency acceptable in real-time settings.**

3

The startup aims to offer **low-cost transcriptions to undercut the market**. However, the high **computational costs of training and running the model** reduce profit margins. Finding a competitive yet profitable pricing strategy requires **extensive experimentation**.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: An Automated Transcription Start-Up

At this stage, Sidney quickly realizes that, even though the models trained in research are a great starting point, it will be a long path toward building a business with a stable product. To build a commercially viable product, there are lots of challenges:

4

Attempts at using new large language models to improve transcripts and adding new features, like automated summaries, run quickly into **excessive costs paid to companies providing the model APIs**. Attempts to self-host open-source large language models were frustratingly difficult due to **brittle and poorly maintained libraries** and ended up needing most of the few high-end GPUs the start-up was able to buy, which were all desperately needed already for other training and inference tasks.

5

Previously focused on data science, the team now faces challenges in building a **user-friendly website for uploading and processing audio files**. Lacking experience and interest in web development, the initial site is clunky and outdated. They also struggle with integrating **payment processing, maintaining uptime, and ensuring secure data storage**. While hiring a front-end developer improved the site's appearance, **communication between the founders and engineers** is difficult due to their different backgrounds, making it hard to align the model, back end, and UI into a product.

6

Model training relies on manual steps and scripts, **making updates time-consuming and error-prone**. Every model improvement or new domain requires extensive re-training, troubleshooting, and hyperparameter tuning. **Fear of breaking the system** has led to not update TensorFlow libraries.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: An Automated Transcription Start-Up

At this stage, Sidney quickly realizes that, even though the models trained in research are a great starting point, it will be a long path toward building a business with a stable product. To build a commercially viable product, there are lots of challenges:

7

After a rough start, customer feedback is mostly positive, and sign-ups are increasing. However, **some customers remain dissatisfied**, reporting serious transcription errors. One complained about incorrect medical diagnoses, while another highlighted poor accuracy for African American Vernacular English.

Debugging is difficult, with every fix uncovering new issues. The team lacks real-time insight into model performance and **relies on customer complaints to identify problems**. Only now are they starting to track basic customer retention metrics.

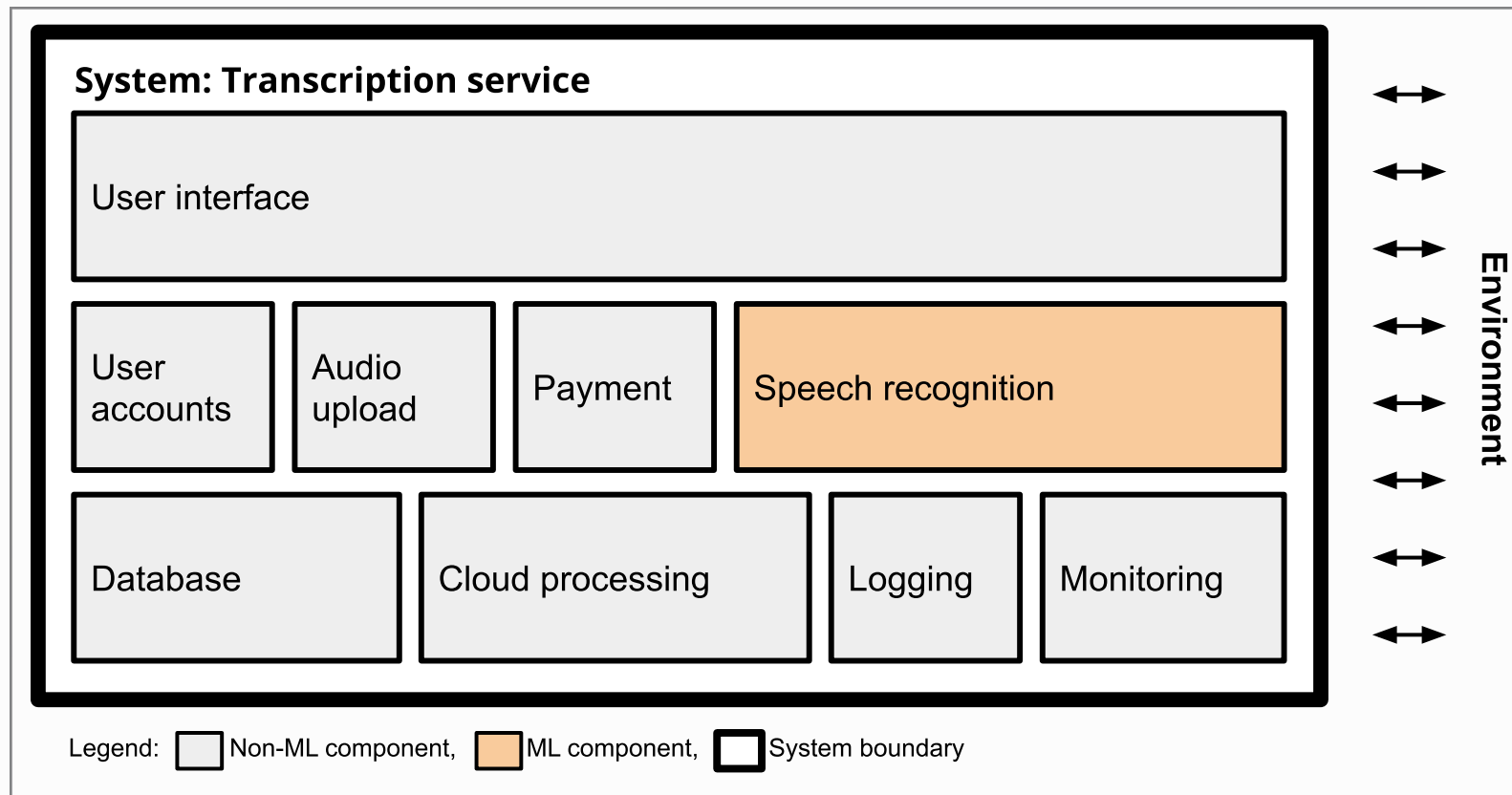
Most of these challenges are probably not very surprising and most are not unique to projects using machine learning. Still, this example illustrates how going from an academic prototype showing the feasibility of a model to a product operating in the real world is far from trivial and requires substantial engineering skills.

For the transcription service, the machine-learned model is clearly the essential core of the entire product. Yet, this example illustrates how, when building a production-ready system, there are many more concerns beyond training an accurate model.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: An Automated Transcription Start-Up

In particular, to turn the model into a product, many other (usually non-ML) parts are needed, such as (a) a user interface to create user accounts, upload audio files, and show results, (b) a data storage and processing infrastructure to queue and store transcriptions and process them at scale, (c) a payment service, and (d) monitoring infrastructure to ensure the system is operating within expected parameters.



In addition, the example did not discuss the the risks of misleading transcriptions, such as medical misinterpretations harming patients, inaccurate legal transcripts affecting court cases, and biases disadvantaging certain dialects. Additionally, privacy concerns around handling sensitive audio data could lead to legal liabilities, regulatory scrutiny, and reputational damage.

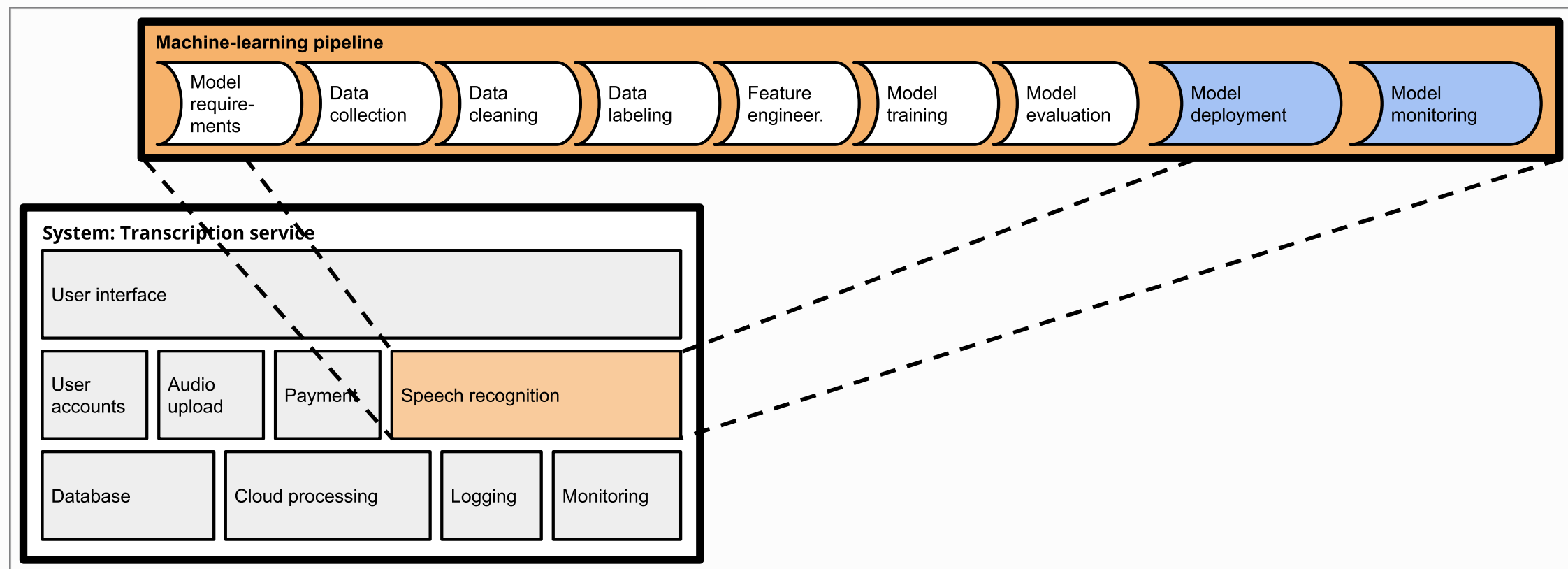
All these insights show that, again, just focusing on model accuracy does not make any sense, especially in cases where a 1-point accuracy improvement does not translate to real-world reliability, user trust, or ethical soundness.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: An Automated Transcription Start-Up

This example also emphasized the critical importance of gathering an appropriate understanding of user needs and system constraints, whether they are related to costs, usability, ethical concerns, or technical limitations.

In the first place, by zooming in the speech recognition component, we would see that *the pipeline starts with model requirements* and ends with deploying the model as a reliable and scalable service.



Secondly, we see that the ML component has to interact with non-ML components; this requires coordination and an understanding of the overall user needs.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: System Thinking

Given how machine learning is part of a larger system, it is important to pay attention to the entire system, not just the machine-learning components. We need a system-wide approach with an interdisciplinary team that involves all stakeholders.

Systems thinking is the name for a discipline that focuses on how systems interact with the environment and how components within the system interact.

System. A set of *inter-related components* that work together in a particular environment to perform whatever functions are required to achieve the system's goal.

System thinking postulates that everything is interconnected, that combining parts often leads to new emergent behaviors that are not apparent from the parts themselves, and that it is essential to understand the dynamics of a system in an environment where actions have effects and may form feedback loops.

AI-Enabled System. A software system that integrates at least one AI component to enhance its functionality, decision-making, or automation capabilities.

AI Component. A specific module or subsystem within a larger system that leverages artificial intelligence techniques to perform a particular function.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: System Thinking

Given how machine learning is part of a larger system, it is important to pay attention to the entire system, not just the machine-learning components. We need a system-wide approach with an interdisciplinary team that involves all stakeholders.

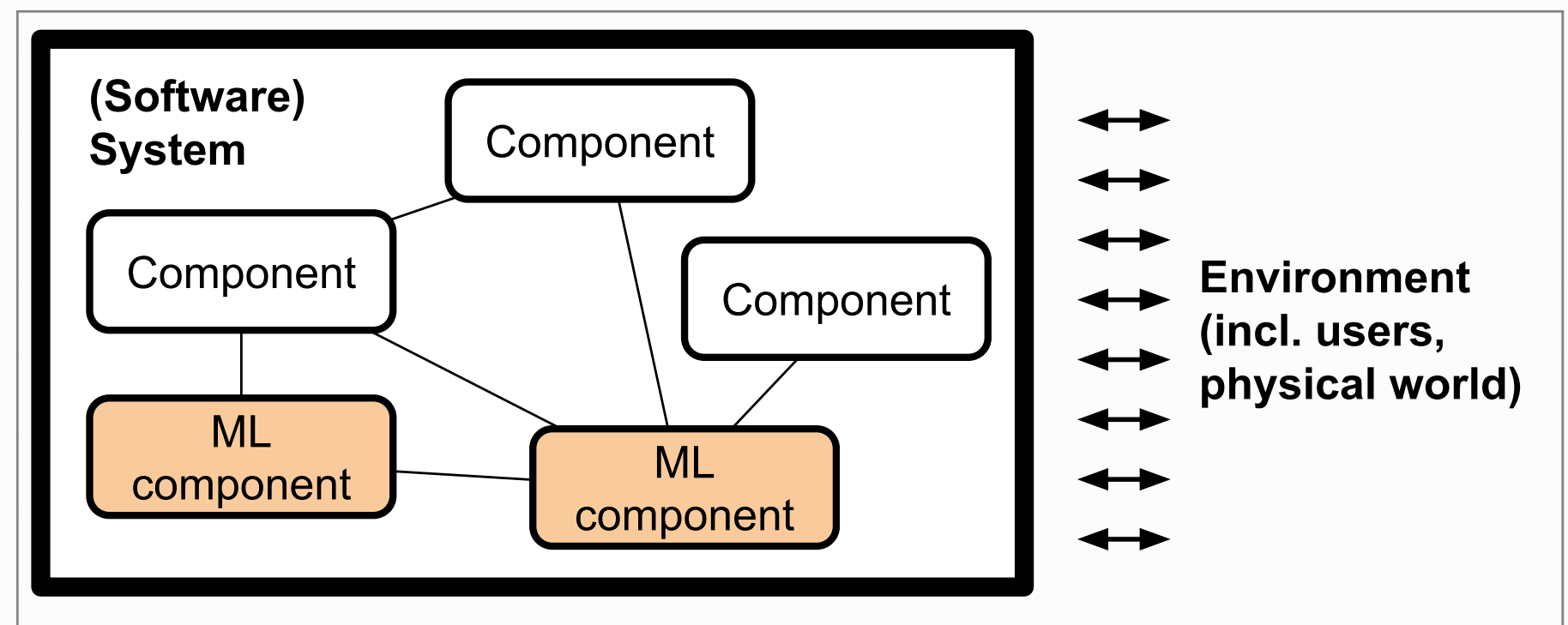
Systems thinking is the name for a discipline that focuses on how systems interact with the environment and how components within the system interact.

particular
the system's goal.

combining parts
the parts
a system in an
ops.

AI component to
es.

tem that leverages



A representation of an AI-enabled system that implements a system thinking approach

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: System Quality versus Model Quality

Outside of machine-learning education and research, model accuracy is almost never a goal in itself, but a means to support the goal of a system or a goal of the organization building the system. A common system goal is to satisfy some user needs, often grounded in an organizational goal of making money.

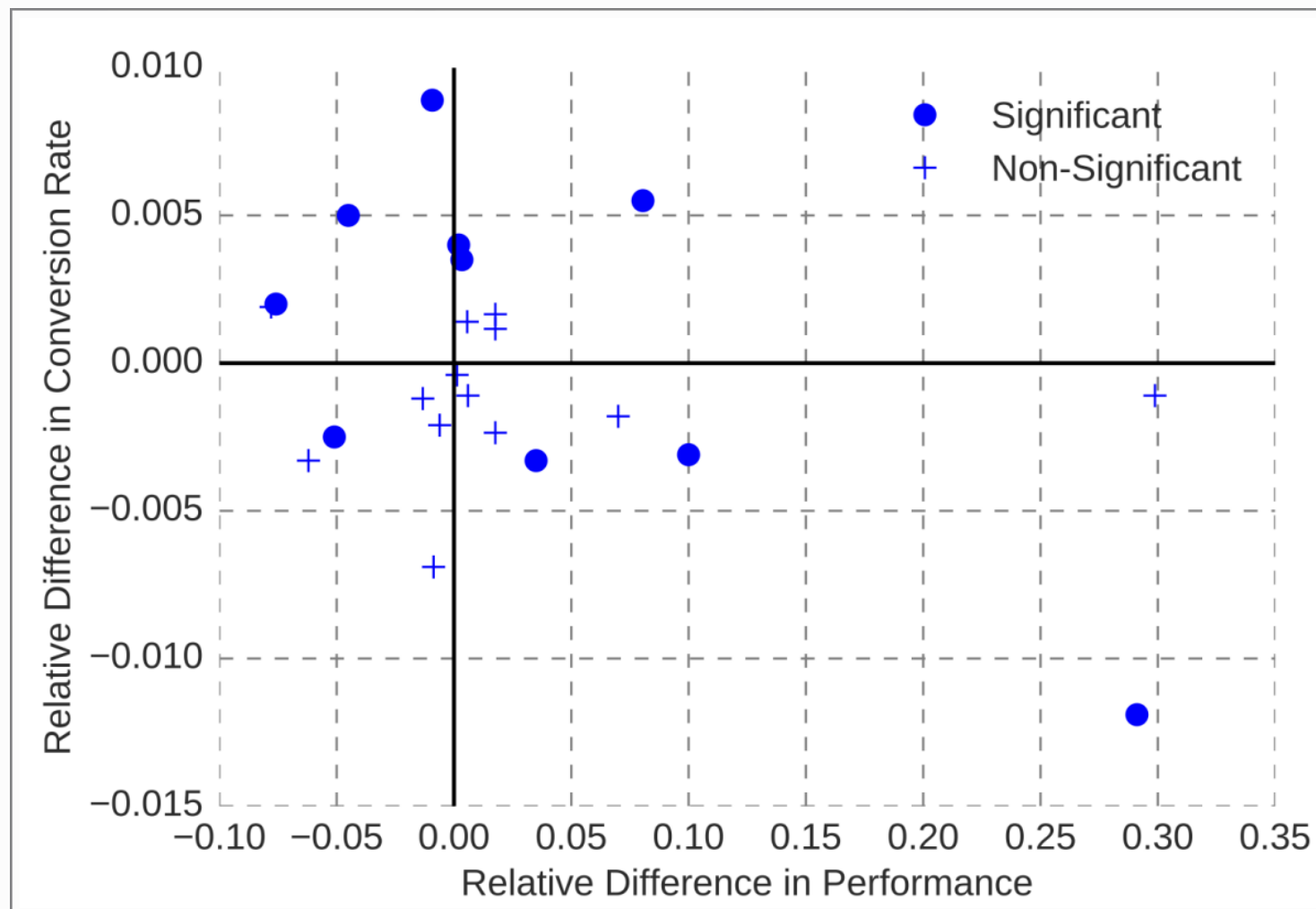
The accuracy of a machine-learned model can *directly* or *indirectly* support such a system goal. For example, a better audio transcription model is more useful to users and might attract more users paying for transcriptions, and predicting the audit risk in tax software may provide value to the users and may encourage sales of additional services. In both cases, **the system goal is distinct from the model's accuracy** but supported more or less directly by it.

Interestingly, improvements in model accuracy do not even have to translate to improvements in system goals. For example, let's consider the experience at the hotel booking site Booking.com.

They showed that improving the accuracy of models predicting different aspects of a customer's travel preferences used for guiding hotel suggestions does not necessarily improve hotel sales—and in some cases, improved accuracy even impacted sales negatively.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: System Quality versus Model Quality



Model accuracy improvement (“Relative Difference in Performance”) does not necessarily translate to improvements in system outcomes (“Conversion Rate”).

It is not always clear why this happens, as there is no direct causal link between model accuracy and sales.

One possible explanation for such observations offered by the team was that **the model becomes too good up to a point where it becomes creepy**: it seems to know too much about a customer’s travel plans when they have not actively shared that information. In the end, more accurate models were not adopted if they did not contribute to overall system improvement.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: System Quality versus Model Quality

Accurate predictions are important for many uses of AI in production systems, but it is not always the most important priority. Accurate predictions may not be critical for the system goal and *“good enough” predictions may actually be good enough.*

In other cases, marginally better model accuracy may come at **excessive costs**, for example, from acquiring or labeling more data, from longer training times, and from privacy concerns—a simpler, cheaper, less accurate model might often be preferable considering the entire system.

Finally, other parts of the system can mitigate problems from inaccurate predictions, such as a **better user interface** that explains predictions to users, **mechanisms for humans to correct mistakes**, or **system-level non-ML safety features**.

From a software-engineering perspective, it is prudent to consider **every machine-learned model as an unreliable function** within a system that sometimes will return unexpected results. Since we must accept eventual mistakes, it is up to other parts of the system, including user interaction design or safety mechanisms, to compensate for such mistakes.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: System Quality versus Model Quality

action systems, but it may not be critical for *good enough*.

excessive costs, for long times, and from it often be

accurate predictions, **mechanisms for features**.

der **every machine-** sometimes will return is up to other parts anisms, to



Consider Geoff Hulten's example of a smart toaster that uses sensors and machine learning to decide how long to toast some bread, achieving consistent outcomes to the desired level of toastedness. Even a highly accurate machine-learned model may eventually suggest toasting times that would burn the toast or even start a kitchen fire.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: System Quality versus Model Quality

While the software is not unsafe itself, the way it actuates the heating element of the toaster could be a safety hazard. More training data and better machine-learning techniques may make the model more accurate and robust and reduce the rate of mistakes, but it will not eliminate mistakes entirely.

Hence, the system designer should look at means to make the system safe even despite the unreliable machine-learning component.

For the toaster, safety mechanisms could include (1) non-ML code to cap toasting time at a maximum duration, (2) an additional non-ML component using a temperature sensor to stop toasting when the sensor readings exceed a threshold, or (3) installing a thermal fuse (a cheap hardware component that interrupts power when it overheats) as a non-ML, non-software safety mechanism.

With these safety mechanisms, the toaster may still occasionally burn some toast when the machine-learned model makes mistakes, but it will not burn down the kitchen :)

Consequences of predictions also manifest in feedback loops. As people react to a system's predictions at scale, we may observe that predictions of the model reinforce or amplify behavior the models initially learned, thus producing more data to support the model's predictions.

Software Engineering for Artificial Intelligence: Requirements Engineering

From Models to Systems: System Quality versus Model Quality

Feedback loops can be positive, such as in public health campaigns against smoking when people adjust behavior in intended ways, providing role models for others and more data to support the intervention.

However, many feedback loops are negative, reinforcing bias and bad outcomes. For example, a system predicting more crime in an area overpoliced due to historical bias may lead to more policing and more arrests in that area—this then provides additional training data reinforcing the discriminatory prediction in future versions of the model even if the area does not have more crime than others.

Just as safety is a system-level property that requires understanding how the software interacts with the environment, so are many other qualities of interest, including security, privacy, fairness, accountability, energy consumption, and user satisfaction.

In the machine-learning community, many of these qualities are now discussed under the umbrella of **responsible AI**. A model-centric view that focuses only on analyzing a machine-learned model without considering how it is used in a system **cannot make any assurances about system-level qualities** such as safety and will have difficulty anticipating feedback loops. **Responsible engineering requires a system-level approach.**

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Definition

A system-level approach inevitably starts with requirements engineering.

Requirement. A statement, typically developed in natural language, that provides indications on a system property and its operational constraints.

Requirements Engineering. The systematic process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed.

In essence, the process aims at investigating, identifying, communicating, and documenting the properties of a system.

FR-1: The system shall allow users to upload audio files up to 2 hours in length for batch processing.

Unique identifier

A clear and concise statement of what the system must do or a condition it must meet.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Definition

A system-level approach inevitably starts with requirements engineering.

Requirement. A statement, typically developed in natural language, that provides indications on a system property and its operational constraints.

Requirements Engineering. The systematic process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed.

In essence, the process aims at investigating, identifying, communicating, and documenting the properties of a system.

User Story.

As a medical professional, I want to upload audio files up to 2 hours in length so that I can transcribe long conversations, interviews, or consultations efficiently without breaking them into smaller parts.

Who

The actor, who is the main user interested in performing a specific task.

What

The action, the activity to perform using the system under development.

Why

The benefits, the advantages the actor has from the feature.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Definition

A system-level approach inevitably starts with requirements engineering.

Requirement. A statement, typically developed in natural language, that provides indications on a system property and its operational constraints.

Requirements Engineering. The systematic process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed.

In essence, the process aims at investigating, identifying, communicating, and documenting the properties of a system.

Choosing between traditional requirements and user stories depends on the nature of the project, team structure, and development approach.

Nowadays, user stories are more diffused for various reasons:

- (1) Most teams follow agile methods, which explicitly favor user stories;
- (2) User stories are great when requirements evolve frequently, like in the case of AI-enabled systems - they capture user needs as they emerge;
- (3) Their focus is on user experience rather than system specifications;

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: When (Not) to Use Artificial Intelligence

Before understanding how to perform requirements engineering, let's first reason on when (not) to use artificial intelligence - this is really the mandatory first step!

Sometimes, AI adoption seems more driven by an interest in the technology or marketing, rather than a real need to solve a specific problem. Adopting AI when not needed may be a bad idea, as machine learning brings a lot of **complexity**, **costs**, and **risks to a system**, which can be avoided when simpler options suffice.

In general, if it is possible to avoid using artificial intelligence and use hand-coded algorithms instead, **it is very often a good idea to do so**. However, there are certain classes of problems for which machine learning seems worth the effort:

1 *Intrinsically hard problems*

For problems we simply do not know how to solve programmatically, artificial intelligence can discover patterns and strategies to solve them regardless.

Tasks that mirror human perception tend to be hard, such as natural language understanding, identifying speech in audio and objects in images, or predicting music preferences. These tasks are complex, and we usually do not fully understand how the original perception works.

Pre-AI attempts to program solutions for such tasks have usually made only limited progress. Machine learning may not work for all intrinsically hard problems, but it can be worth a try.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: When (Not) to Use Artificial Intelligence

Before understanding how to perform requirements engineering, let's first reason on when (not) to use artificial intelligence - this is really the mandatory first step!

Sometimes, AI adoption seems more driven by an interest in the technology or marketing, rather than a real need to solve a specific problem. Adopting AI when not needed may be a bad idea, as machine learning brings a lot of **complexity**, **costs**, and **risks to a system**, which can be avoided when simpler options suffice.

In general, if it is possible to avoid using artificial intelligence and use hand-coded algorithms instead, **it is very often a good idea to do so**. However, there are certain classes of problems for which machine learning seems worth the effort:

2 *Big problems*

For some problems, hand-crafting a program to solve a problem might be possible, but it would be so complex and large that manually maintaining it becomes infeasible.

Resolving conflicts between multiple hard-coded rules and handling exceptions can be especially tedious. For example, we might attempt to manually encode music recommendations, but there are so many artists and tracks that rule-based heuristics might become unmaintainable.

Manually curating directories of websites has been tried in the early days of the internet, but it simply did not scale, whereas (AI-based) search engines have dominated ever since.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: When (Not) to Use Artificial Intelligence

Before understanding how to perform requirements engineering, let's first reason on when (not) to use artificial intelligence - this is really the mandatory first step!

Sometimes, AI adoption seems more driven by an interest in the technology or marketing, rather than a real need to solve a specific problem. Adopting AI when not needed may be a bad idea, as machine learning brings a lot of **complexity**, **costs**, and **risks to a system**, which can be avoided when simpler options suffice.

In general, if it is possible to avoid using artificial intelligence and use hand-coded algorithms instead, **it is very often a good idea to do so**. However, there are certain classes of problems for which machine learning seems worth the effort:

3 *Time-Changing problems*

For problems where the inputs and solutions change frequently, machine learning may be able to more easily keep up if suitable data is available.

For example, in music recommendations, individual preferences and music trends change over time, as does what music is available. Change is constant, and even if we had hardcoded recommendation rules, it would be tedious to update them regularly.

For such time-changing problems, building a (complex) ML-based solution that can automatically update the system may be easier.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: When (Not) to Use Artificial Intelligence

In addition, AI should only be used in applications that can tolerate mistakes. In essence, AI is an unreliable function that often works but sometimes makes mistakes. It can even be hard to define what it would mean for them to be correct in the first place...

In some settings, mistakes are simply *acceptable*. For example, music recommendations are not critical, and subscribers can likely tolerate (some) poor recommendations and benefit from recommendations even if not all are equally good.

However, be aware that also seemingly harmless predictions *can cause harm*, for example music recommendation systematically discriminating against Hispanic subscribers or LGBTQ+ artists.

So, in general, **if the correctness of a component is essential, artificial intelligence is not a good match**, especially if there is a specification of correct behavior that we can implement with traditional code

For example, we would not want to use artificial intelligence components when tabulating information in accounting systems or when transmitting control signals in an airplane, where we can specify the correct expected behavior precisely, and where behaving correctly is crucial.

In the end, a decision on whether to use artificial intelligence for a problem comes down to comparing **costs and benefits**.

Requirements Engineering: Setting and Measuring Goals

Establishing high-level project goals is usually one of the first steps in eliciting the requirements for the system. Goals may be revisited regularly as requirements are collected, solutions are designed, or the system is observed in production.

Goals are also useful for the design process: when decomposing a system and assigning responsibilities to components, we can identify component goals and ensure they align with the overall system goals.

To untangle different goals, it is useful to question goals at different layers and to discuss how different goals relate to each other.

1 *Organization Goals*

The most general goals are usually at the organizational level. Aside from nonprofits, organizational goals almost always relate to money: revenue, profit, or stock price. Nonprofit organizations often have clear goals as part of their charter, such as increasing animal welfare, reducing CO₂ emissions, and curing diseases.

Since organizational objectives are often high-level and pursue long-term goals that are difficult to measure now, often leading indicators are used as more readily observable proxy measures that are expected to correlate with future organizational success.

Many organizations frame leading indicators as **key performance indicators** (KPIs).

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Setting and Measuring Goals

Establishing high-level project goals is usually one of the first steps in eliciting the requirements for the system. Goals may be revisited regularly as requirements are collected, solutions are designed, or the system is observed in production.

Goals are also useful for the design process: when decomposing a system and assigning responsibilities to components, we can identify component goals and ensure they align with the overall system goals.

To untangle different goals, it is useful to question goals at different layers and to discuss how different goals relate to each other.

2 *Product Goals*

Users typically use a software product with a specific goal. In many cases, we have multiple different kinds of users with different goals.

In addition to users who directly interact with the product, there are often also people who are indirectly affected by the product or who have expectations of the product.

For instance, in the AI-powered transcription service presented earlier, indirect affected people might be the interviewees (if the AI misinterprets speech, their words might be inaccurately represented) or editors (they rely on accurate transcriptions for fact-checking and publishing).

Requirements Engineering: Setting and Measuring Goals

Establishing high-level project goals is usually one of the first steps in eliciting the requirements for the system. Goals may be revisited regularly as requirements are collected, solutions are designed, or the system is observed in production.

Goals are also useful for the design process: when decomposing a system and assigning responsibilities to components, we can identify component goals and ensure they align with the overall system goals.

To untangle different goals, it is useful to question goals at different layers and to discuss how different goals relate to each other.

3 *Model Goals*

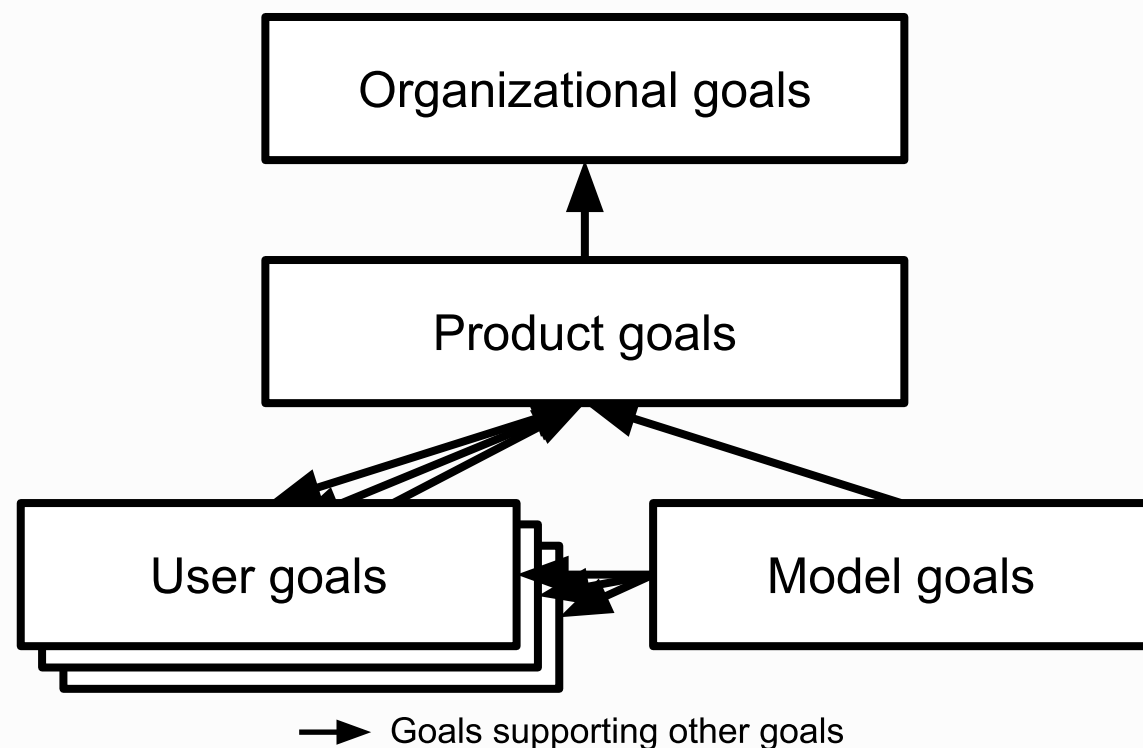
From the perspective of an artificial intelligence component, the goal is almost always to optimize some notion of accuracy of predictions.

Model quality can be measured offline with test data and approximated in production with telemetry, as we will discuss in the context of this course.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Setting and Measuring Goals

Goals at the different levels are usually not independent. For instance, if product and user goals align, then a product that better meets its goals makes users happier, and users may be more willing to cooperate with the product. Similarly, better models hopefully make our users happier or contribute in various ways to making the product achieve its goals.



We can therefore see the goals to be interconnected, with each of them influencing the others.

It is usually a good idea to identify goals at all levels and understand how they relate to each other. It is particularly important to contextualize the product and model goals in the context of the goals of the organization and the various users.

Balancing conflicting goals may require some deliberation and negotiation that are normal during requirements engineering.

Software Engineering for Artificial Intelligence: Requirements Engineering

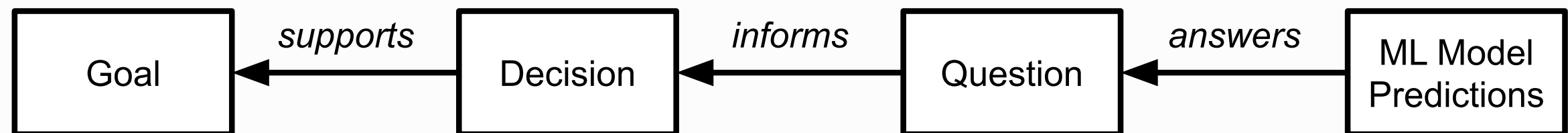
Requirements Engineering: Setting and Measuring Goals

Goals are high-level requirements from which often more low-level requirements are derived. To understand the requirements of a software product, understanding the goals for the product and the goals of its creators is an important early step.

In addition, goals can help identify opportunities for AI in an organization.

Understanding the goals of an organization and the goals of individuals within it, we can (a) explore the decisions that people routinely make within the organization to further their goals, (b) ask which of those decisions could be supported by predictions, and (c) which of those predictions can be supported with artificial intelligence.

For example, we can collect user stories in the form of “*As [role], I need to make [decision] to achieve [goal]*” and “*As [role], I need to know [question] to make [decision]*”. That is, goals are an excellent bridge to explore the business case of artificial intelligence and what kind of models and software products to prioritize and whether to support humans with predictions or fully automate some tasks.



Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: What?

Requirements Engineering. The process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed.

We are used to classify requirements in three categories:

- (1) Functional requirements.** Statements of services, how the system should react to particular inputs, what functionalities should be provided. Functional requirements are not concerned with how these functions should be achieved, just what is to be achieved.
- (2) Non-functional requirements.** Statements of the attributes and properties of the software, rather than the functions. Non-functional requirements are not concerned with what functions are to be achieved, just how these should be achieved.
- (3) Domain requirements.** Constraints that the customer imposes on the environment in which the system must operate. These are also called pseudo-requirements.

In traditional systems, you used to classify non-functional requirements in terms of maintainability, performance, security, reliability, and safety aspects.

In AI-intensive systems, the discussion is quite similar. Yet, there is a key difference: in the vast majority of the cases, these systems fail or get abandoned because of violations of non-functional properties —> non-functional requirements are much more important, e.g., see the Amazon hiring tool example.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: A New Set of Non-Functional Requirements?

Some non-functional requirements are indeed different, others may assume a different meaning or have different implications.

Quality

The ability of an AI component to be sound, stable, and reliable.

Examples: Accuracy, retrainability, testability, usability, interpretability, flexibility, explainability.

Infrastructure

The ability to build, run, and manage AI components.

Examples: Scalability, performance, fault tolerance, efficiency.

Ethics

The ability of an AI component to act respecting laws and ethics.

Examples: Fairness, transparency, trust, bias.

Safety

The ability of an AI component not to cause harm to externals.

Examples: Privacy, data access.

Security

The ability of an AI component to secure the data it manages.

Examples: Integrity, data issues.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: A New Set of Non-Functional Requirements?

Some non-functional requirements are indeed different, others may assume a different meaning or have different implications.

Quality

Examples: Accuracy, flexibility, explainability

In traditional software engineering, source code quality is included within maintainability. Here's quality has a much higher weight, as it encompasses all major properties of AI!

able, and reliable.

stability,

Infrastructure

The ability to build, run, and manage AI components.

Examples: Scalability, performance, fault tolerance, efficiency.

Ethics

The ability of an AI component to act respecting laws and ethics.

Examples: Fairness, transparency, trust, bias.

Safety

The ability of an AI component not to cause harm to externals.

Examples: Privacy, data access.

Security

The ability of an AI component to secure the data it manages.

Examples: Integrity, data issues.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: A New Set of Non-Functional Requirements?

Some non-functional requirements are indeed different, others may assume a different meaning or have different implications.

Quality

The ability of an AI component to be sound, stable, and reliable.

Examples: Accuracy, retrainability, testability, usability, interpretability, flexibility, explainability.

Infrastructure

Examples: Scalability

As we will discuss many times, AI is all about data processing! Certain data (e.g., images) are hard to process; Certain AI models (e.g., deep learning) are hard to train and execute.

nents.

Ethics

The ability of an AI component to act respecting laws and ethics.

Examples: Fairness, transparency, trust, bias.

Safety

The ability of an AI component not to cause harm to externals.

Examples: Privacy, data access.

Security

The ability of an AI component to secure the data it manages.

Examples: Integrity, data issues.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: A New Set of Non-Functional Requirements?

Some non-functional requirements are indeed different, others may assume a different meaning or have different implications.

Quality

The ability of an AI component to be sound, stable, and reliable.

Examples: Accuracy, retrainability, testability, usability, interpretability, flexibility, explainability.

Infrastructure

The ability to build, run, and manage AI components.

Examples: Scalability, performance, fault tolerance, efficiency.

Ethics

The ability of an AI component to comply with laws and ethics.

Examples: Fairness, transparency, accountability.

Ethical AI is becoming the next big challenge. The European Commission is pushing billions of euros to grant projects able to assess and improve ethical aspects of AI —> Consider this aspects for your Master thesis if you would like to impress your future boss!

Safety

The ability of an AI component to avoid harm to externals.

Examples: Privacy, data access.

Security

The ability of an AI component to secure the data it manages.

Examples: Integrity, data issues.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: A New Set of Non-Functional Requirements?

Some non-functional requirements are indeed different, others may assume a different meaning or have different implications.

Quality

The ability of an AI component to be sound, stable, and reliable.

Examples: Accuracy, retrainability, testability, usability, interpretability, flexibility, explainability.

Infrastructure

The ability to build, run, and manage AI components.

Examples: Scalability, performance, fault tolerance, efficiency.

Ethics

The ability of an AI component to act respecting laws and ethics.

Examples: Fairness, transparency, trust, bias.

Safety

T

AI might be extremely powerful (e.g., self-driving cars) and must therefore being regulated and carefully controlled against the potential harm to externals.
harm that it can cause.

Examples: Privacy,

Security

The ability of an AI component to secure the data it manages.

Examples: Integrity, data issues.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: A New Set of Non-Functional Requirements?

Some non-functional requirements are indeed different, others may assume a different meaning or have different implications.

Quality

The ability of an AI component to be sound, stable, and reliable.

Examples: Accuracy, retrainability, testability, usability, interpretability, flexibility, explainability.

Infrastructure

The ability to build, run, and manage AI components.

Examples: Scalability, performance, fault tolerance, efficiency.

Ethics

The ability of an AI component to act respecting laws and ethics.

Examples: Fairness, transparency, trust, bias.

Safety

The ability of an AI component not to cause harm to externals.

Examples: Privacy,

Security

The ability of an AI component to protect the data it manages.

Examples: Integrity, data issues.

The sneakiest trick of modern security threats is the ability to inject insecure data or objects within AI models in order to destabilize it, causing potential issues.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: A New Set of Non-Functional Requirements?

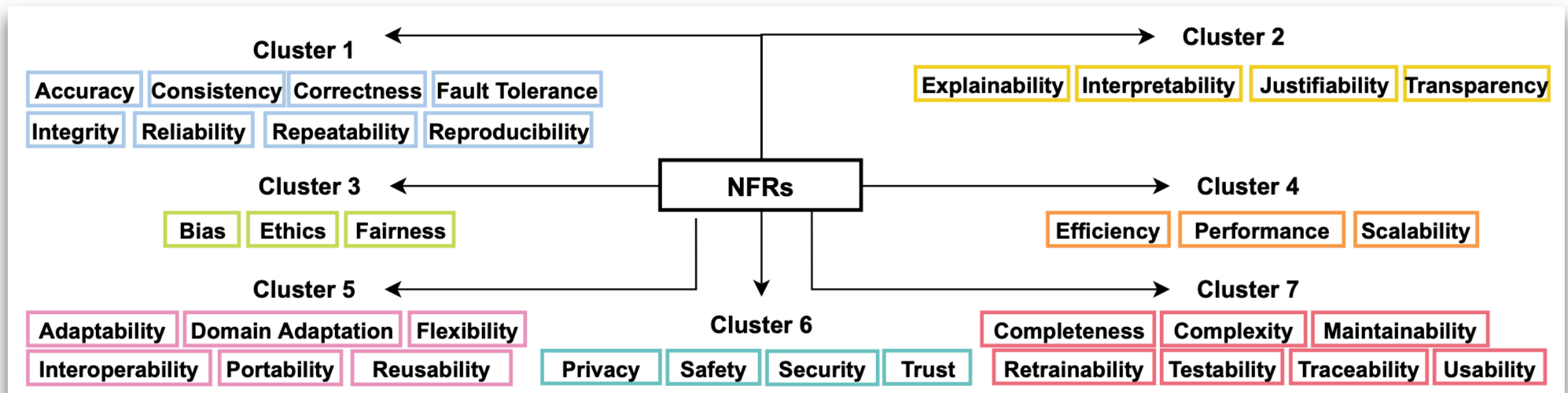
Yet, AI-intensive systems might be considered under multiple additional perspectives!

NFRs	Definition
Accuracy	The number of correctly predicted data points out of all the data points.
Adaptability	The ability of a system to work well in different but related contexts.
Bias	A phenomenon that occurs when an algorithm produces results that are systematically prejudiced due to erroneous assumptions in the ML process.
Completeness	An indication of the comprehensiveness of available data, as a proportion of the entire data set, to address specific information requirements.
Complexity	When a system or solution has many components, interrelations or interactions, and is difficult to understand.
Consistency	A series of measurements of the same project carried out by different raters using the same method should produce similar results.
Correctness	The output of the system matches the expectations outlined in the requirements, and the system operates without failure.
Domain Adaptation	The ability of a model trained on a source domain to be used in a different—but related—domain.
Efficiency	The ability to accomplish something with minimal time and effort.
Ethics	Concerned with adding or ensuring moral behaviors.
Explainability	The extent to which the internal mechanics of ML-enabled system can be explained in human terms.
Fairness	The ability of a system to operate in a fair and unbiased manner
Fault Tolerance	The ability of a system to continue operating without interruption when one or more of its components fail.
Flexibility	The ability of a system to react to changing demands or conditions.
Integrity	The ability to ensure that data is real, accurate and safeguarded from unauthorised modification.
Interpretability	The extraction of relevant knowledge from a model concerning relationships either contained in data or learned by the model
Interoperability	The ability for two systems to communicate effectively
Justifiability	The ability to be show the output of an ML-enabled system to be right or reasonable.
Maintainability	The ease with which a system or component can be modified to correct faults, improve performance or other attributes, or adapt to a changed environment
Performance	The ability of a system to perform actions within defined time or throughput bounds.
Portability	The ability to transfer a system or element of a system from one environment to another.
Privacy	An algorithm is private if an observer examining the output is not able to determine whether a specific individual's information was used in the computation.
Reliability	The probability of the software performing without failure for a specific number of uses or amount of time.
Repeatability	The variation in measurements taken by a single instrument or person under the same conditions.
Retrainability	The ability to re-run the process that generated the previously selected model on a new training set of data.
Reproducibility	One can repeatedly run your algorithm on certain datasets and obtain the same (or similar) results.
Reusability	The ability of reusing the whole or the greater part of the system component for similar but different purpose.
Safety	The absence of failures or conditions that render a system dangerous
Scalability	The ability to increase or decrease the capacity of the system in response to changing demands.
Security	Security measures ensure a system's safety against espionage or sabotage.
Testability	The ability of the system to to support testing by offering relevant information or ensuring the visibility of failures.
Transparency	The extent to which a human user can infer why the system made a particular decision or produced a particular externally-visible behaviour.
Traceability	The ability to trace work items across the development lifecycle.
Trust	A trusted system is a system that is relied upon to a specified extent to enforce a specified security, or a security policy.
Usability	How effectively users can learn and use a system.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: A New Set of Non-Functional Requirements?

Yet, AI-intensive systems might be considered under multiple additional perspectives! The management of non-functional requirements is therefore quite complicated and, as we will see, this represents an optimization problem. Researchers and practitioners are currently investigating methods to cluster requirements based on shared characteristics, so that they can be more easily treated.

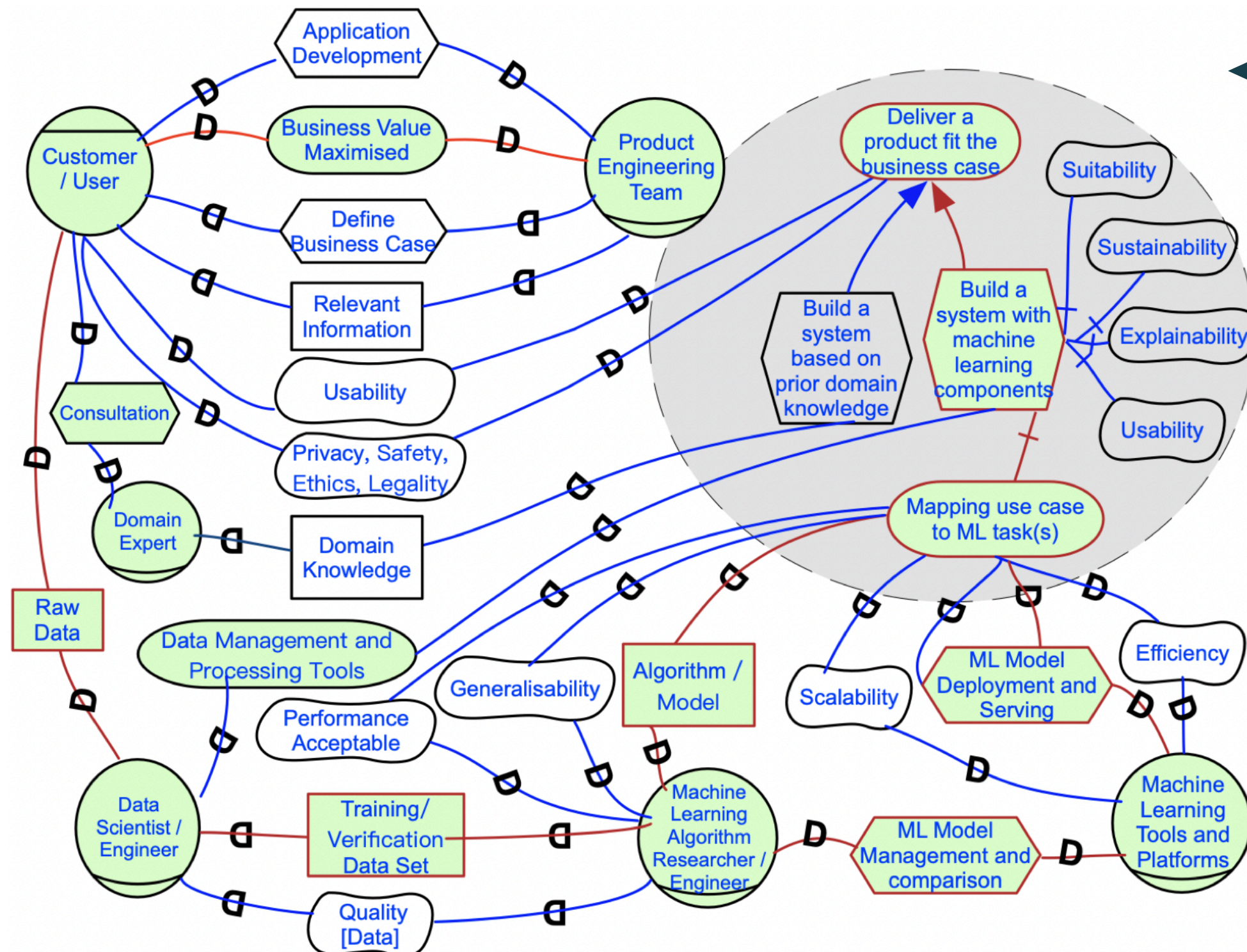


This clustering has key implications. For instance, if we know that bias, ethics, and fairness share common characteristics, software analytics techniques and methods working on the analysis and optimization of one of them (e.g., fairness) might implicitly be useful for other non-functional requirements (e.g., bias and ethics). In the course, we will discuss some methods to optimize non-functional requirements while keeping the overall accuracy of AI models.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Increasing the Complexity

Let's take a quick look at the collaborations required to define requirements...

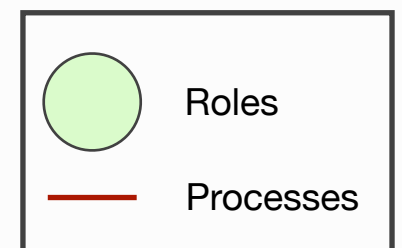


Identifying requirements is an effort!

It is mainly a human activity
—> it may be wrong!

Automation, also
in this case, may
be relevant!

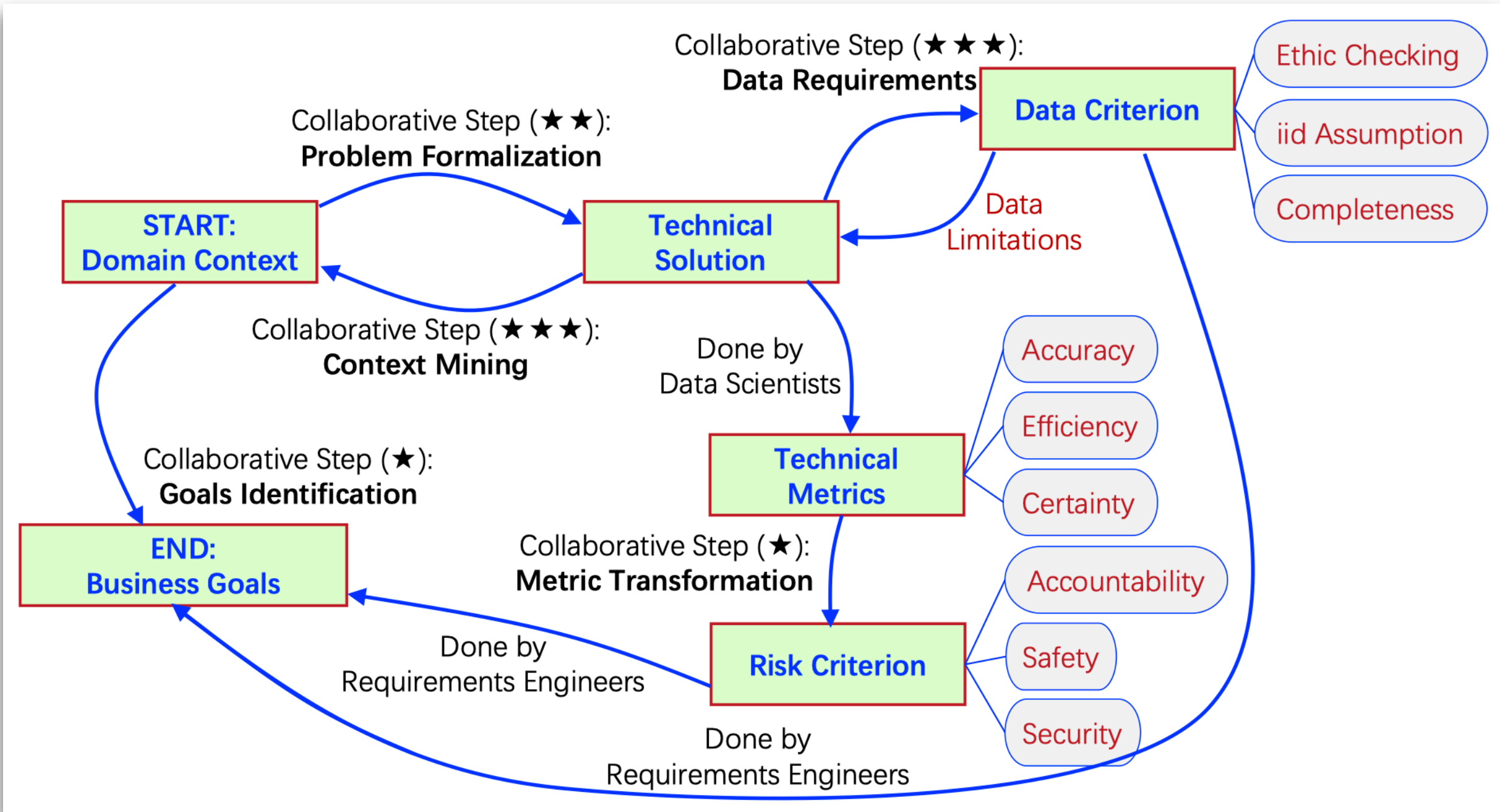
Consider the case
of automatic
extraction of non-
functional
requirements!



Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Increasing the Complexity

In simplified terms, this is the internal set of collaborative steps required to define the business goals and use cases of an AI-based application.



Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Increasing the Complexity

Requirements Engineering for Machine Learning: Perspectives from Data Scientists

Andreas Vogelsang
Technische Universität Berlin
Berlin, Germany
andreas.vogelsang@tu-berlin.de

Markus Borg
RISE Research Institutes of Sweden AB
Lund, Sweden
markus.borg@ri.se

Abstract—Machine learning (ML) is used increasingly in real-world applications. In this paper, we describe our ongoing endeavor to define characteristics and challenges unique to Requirements Engineering (RE) for ML-based systems. As a first step, we interviewed four data scientists to understand how ML experts approach elicitation, specification, and assurance of requirements and expectations. The results show that changes in the development paradigm, i.e., from coding to training, also demands changes in RE. We conclude that development of ML systems demands requirements engineers to: (1) understand ML performance measures to state good functional requirements, (2) be aware of new quality requirements such as explainability, freedom from discrimination, or specific legal requirements, and (3) integrate ML specifics in the RE process. Our study provides a first contribution towards an RE methodology for ML systems.

Index Terms—machine learning, requirements engineering, interview study, data science

I. INTRODUCTION

Machine Learning (ML) has gained much attention in recent years and accomplished major technical breakthroughs. ML success stories include image recognition, natural language processing, and beating humans in complex games [1]. The key enablers for the ongoing ML disruption are based on the combination of the computational power of modern processors (especially GPUs), the availability of large amounts of data, and the accessibility of these complex technologies by rather simple-to-use frameworks and libraries. As a result, many companies use ML to improve their products and processes.

Whether the increasing amount of ML in contemporary software solutions demands requirements engineers to adapt their work is an open question. One might argue that ML is just another technology, i.e., it should not have any influence on the requirements. On the other hand, ML engineering constitutes a paradigm shift compared to conventional software engineering. Andrej Karpathy, Director of AI at Tesla, even refers to the new era as “Software 2.0” – no longer does all behavior emerge from a set of manually coded rules. Instead, most ML approaches generate rules based on a set of examples (training data) and a specific fitness function. In addition, a recent survey suggests that Requirements Engineering (RE) is the most difficult activity for the development of ML-based systems [2].

In this paper, we share results that indicate that RE must evolve to match the specifics of ML systems. Currently, most

decisions in the development of ML systems are made by data scientists. These decisions include the definition of the fitness functions, the selection and preparation of data, and the quality assurance. However, these decisions should be based on an understanding of the business domain and the stakeholder needs. From our perspective, this falls into the profession of a requirements engineer.

We conducted interviews with four data scientists to explore their perceptions on RE. The interviews covered specific requirements for ML systems, challenges involved in RE for ML, and how the RE process needs to evolve. Our main findings are that requirements engineers need to be aware of new requirements types introduced by the ML paradigm, e.g., explainability and freedom from discrimination, and they need to understand quantitative ML measures to specify good functional requirements. We elaborate on our results in Sections IV and V after having presenting background in Section III and the study design in Section III.

II. BACKGROUND AND RELATED WORK

A. Machine Learning and Software Engineering

ML is the practice of getting computers to act without being explicitly programmed, organized in three main types. *Supervised learning* finds mappings between input-output pairs based on labelled examples with the correct answer. This type of ML dominates industrial applications today, but requires high-quality data. *Unsupervised learning* identifies patterns in data without access to any labels. A typical application of this type of ML is clustering. *Reinforcement learning* relies on a reward signal to quantify the ML performance, inspired by the carrot-and-stick principle. Video games have been the primary target for this approach, but software engineering examples exist, e.g., software testing [3] and self-adaptive systems [4].

Using ML to learn system features is fundamentally different from manually implementing them in source code. The predictive power is embedded in the training data, harnessed using a limited number of function calls. State-of-the-art supervised learning use backpropagation to fit millions of parameter weights representing information propagation in deep neural networks, resulting in an ML model. Compared to human-readable source code, ML models are opaque constructs that are intrinsically hard to interpret. Standard practices such as source code reviews and exhaustive coverage testing are not

According to data scientists, among all the requirements, some questions must be prioritized:

- (1) The evaluation metric must be specified;
- (2) Explainability must be ensured. Quoting from the paper: *“Explainability is twofold: On the one side, there is a need to explain the model (what has been learned). On the other side, there is a need to explain single predictions of the model”*.
- (3) ML algorithms amplify discrimination bias. *“Using only logics of discrimination that are societally and legally accepted”* is key.
- (4) Legal aspects are hard to acquire, but key to design ML systems: *“The law states that you must know what your ML model will need to deliver before developing it. As a data scientist, I know that I probably need 20 out of 100 features but I don’t know which 20. I have to ask for consent to use all of them. In the end, the legal department tells me that I’m collecting 80 features I’m not using. And that is illegal”*.
- (5) Data requirements are a new class of requirements!

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How?

Requirements Elicitation

The objective of the elicitation phase is to gain knowledge about the project domain and requirements.

The various sources of domain knowledge include customers, business manuals, the existing software of the same type, standards, and other stakeholders of the project.

This activity is conducted with *qualitative* methods, such as:

- (1) *Interviews*. Individual discussions with the stakeholders involved in the project.
- (2) *Surveys*. Larger-scale questionnaires conducted to gather knowledge from a large amount of customers.
- (3) *Task analysis*. In-situ observations where the analyst observes the current use cases.
- (4) *Prototyping*. Development of preliminary versions of the software to ease the communication with stakeholders.

Requirements Analysis

Requirements Validation

Requirements Management

The elicitation phase does not produce any formal model of the requirements understood. It serves as a way to widen the knowledge of the analyst.

Requirements Engineering: How?

ARTIFICIAL INTELLIGENCE FOR REQUIREMENTS ENGINEERING INTERMEZZO

A Bird's Eye View on Requirements Engineering and Machine learning

1st Tahira Iqbal
fortiss GmbH
Munich, Germany
iqbal@fortiss.org

2nd Parisa Elahidoost
fortiss GmbH
Munich, Germany
elahidoost@fortiss.org

3rd Levi Lúcio
fortiss GmbH
Munich, Germany
lucio@fortiss.org

Abstract—Machine learning (ML) has demonstrated practical impact in a variety of application domains. Software engineering is a fertile domain where ML is helping in automating different tasks. In this paper, our focus is the intersection of software requirement engineering (RE) and ML. To obtain an overview of how ML is helping RE and the research trends in this area, we have surveyed a large number of research articles. We found that the impact of ML can be observed in requirement elicitation, analysis and specification, validation and management. Furthermore, in these categories, we discuss the specific problem solved by ML, the features and ML algorithms used as well as datasets, when available. We outline lessons learned and envision possible future directions for the domain.

Index Terms—Requirements Engineering, Machine learning, State of the art, Overview

I. INTRODUCTION

Machine learning algorithms have been shown to have considerable practical importance in many application domains. This is especially true of domains where large databases are available and a need for exploring some kind of consistency exists or, domains where a program needs to adapt itself to changes [56]. Requirements engineering is a critical part of software engineering and it seems appropriate to use machine learning methods for requirements engineering tasks. Because requirements specification documents are mainly given in natural language, ML can be useful by emulating human processing.

This paper aims to present a survey of how ML benefits existing RE approaches. More precisely, we pursue the following research questions:

RQ1: What is the current state of the practice in ML & RE?

RQ2: What types of learning methods are used when ML is applied to RE?

RQ3: Which are the RE problems that currently use ML methods?

RQ4: Is using ML methods improving RE?

To reply to these research questions, we have performed a literature review, split into data preparation, data collection, and data analysis phase. First, a search string was prepared based on the research questions, then a search was performed over a predefined set of databases and all identified studies were assessed by means of title and abstract. Our literature

review is not meant to be an exhaustive study of the field – rather we are offering a snapshot of the current state-of-the-art by borrowing some techniques from Systematic Literature Reviewing.

The major contributions of this article are as follows:

- We provide an overview of the current state of the art of some of the challenges RE faces that may be handled through ML techniques. We focus on two important aspects:
 - Providing an overview of the ML problem categories (classification, regression, clustering, etc.) in use for the support of RE tasks (elicitation, analysis, validation and management).
 - Providing an overview of the common ML models (decision tree, K-Nearest Neighbors, Naive Bayesian, etc.) for tackling RE problems and the data sets if available.
- We analyse the literature to discover trends and lessons on the use of ML in RE.

The paper is organized as follows. The rest of this section provides background information on ML and RE required to understand the remaining of the paper. Section II provides an overview of RE tasks where ML has been used. In section III we summarize the major findings of our study. Finally, in section IV we state the threats to the validity of our work and section V concludes the paper.

A. Machine Learning

Machine-Learning (ML) [35] is a range of algorithms to approximate discover patterns in data. Historically, models and heuristics are human-built exhaustive prescriptions of how a system should behave. ML is grounded on different premises: rather than relying on humans to input all the possible cases the system can handle, the field attempts to extrapolate patterns from a representative set of examples that illustrates expected behaviors. The way in which a learning algorithm operates attempts to emulate the way in which humans learn: from a set of examples, a general model for a behavior is induced.

Many learning algorithms exist, based on different visions of how learning happens in practice [13]. All these algorithms have in common the notion of *features*. Features correspond to characteristics of what is being learned and provide the

Artificial intelligence itself can support the requirements analysis process: In this case we refer to AI4SE(4AI)!

¹This work was supported in part by the European Union's Horizon 2020 Research and Innovation program under grant agreement no. 674875

Requirements Engineering: How?

ARTIFICIAL INTELLIGENCE FOR REQUIREMENTS ENGINEERING INTERMEZZO

Detecting Privacy Requirements from User Stories with NLP Transfer Learning Models*

Francesco Casillo^a, Vincenzo Deufemia^a and Carmine Gravino^{a,*}

^aDepartment of Computer Science, University of Salerno, Via Giovanni Paolo II, 132, Fisciano(SA), 84084, Italy

ARTICLE INFO

Keywords:
User Stories
Natural Language Processing
Deep Learning
Transfer Learning

ABSTRACT

Context: To provide privacy-aware software systems, it is crucial to consider privacy from the very beginning of the development. However, developers do not have the expertise and the knowledge required to embed the legal and social requirements for data protection into software systems.
Objective: We present an approach to decrease privacy risks during agile software development by automatically detecting privacy-related information in the context of user story requirements, a prominent notation in agile Requirement Engineering (RE).
Methods: The proposed approach combines Natural Language Processing (NLP) and linguistic resources with deep learning algorithms to identify privacy aspects into User Stories. NLP technologies are used to extract information regarding the semantic and syntactic structure of the text. This information is then processed by a pre-trained convolutional neural network, which paved the way for the implementation of a Transfer Learning technique. We evaluate the proposed approach by performing an empirical study with a dataset of 1680 user stories.
Results: The experimental results show that deep learning algorithms allow to obtain better predictions than those achieved with conventional (shallow) machine learning methods. Moreover, the application of Transfer Learning allows to considerably improve the accuracy of the predictions, ca. 10%.
Conclusions: Our study contributes to encourage software engineering researchers in considering the opportunities to automate privacy detection in the early phase of design, by also exploiting transfer learning models.

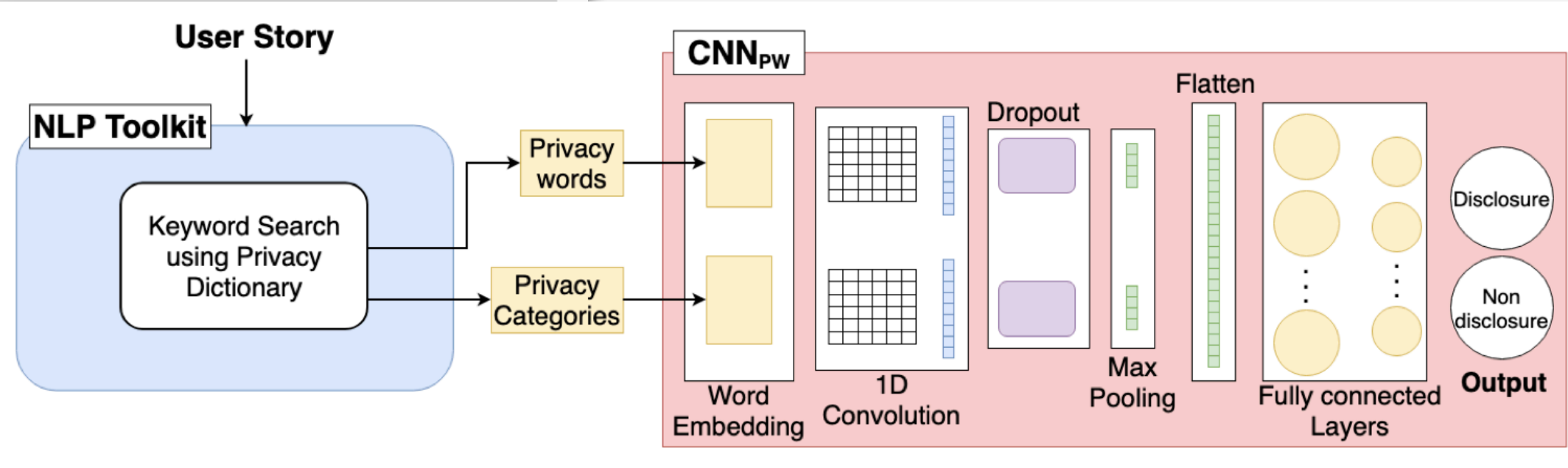
1. Introduction

Requirements engineering is a complex activity of software engineering and imperfections in the requirements lead to design flaws and can cause RE is based on face-to-face interactions and developers who are not aware of the problems, but this does not solve the problem. Among them, the detection of Non-Functional Requirements (NFRs) by stakeholders is often a difficult task for several reasons [3]. To alleviate this problem, several approaches have been proposed [4, 5, 6]. In [4], the authors have proposed a tool-based approach to extract sentences in documents, which are classified into NFR categories [7]. In general, these approaches provide only an overview of the requirements since stakeholders usually have different needs, they might have difficulties in understanding a software application, i.e., they might not be able to express their requirements in a clear and concise way.

*This work has been partially supported by the Italian Ministry of Education, University and Research (MUR) under the project "PATHY: Empowering People in the Digital Age" (Progetti di Rilevante Interesse Nazionale - PRIN 2017MX9T7H).

*Corresponding author
✉ fcasillo@unisa.it (F. Casillo); deufemia@unisa.it (V. Deufemia);
gravino@unisa.it (C. Gravino)
ORCID(s): 0000-0003-4869-8068 (F. Casillo); 0000-0002-6711-3590 (V. Deufemia); 0000-0002-4394-9035 (C. Gravino)

The proposed approach combines the use of linguistic resources and Natural Language Processing (NLP) techniques to extract features useful not only to capture the semantic meaning and the syntactic structure of the text, but also to determine the presence or absence of privacy-related words.



Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How?

ARTIFICIAL INTELLIGENCE FOR REQUIREMENTS ENGINEERING INTERMEZZO

ReFAIR: Toward a Context-Aware Recommender for Fairness Requirements Engineering

Carmine Ferrara 
University of Salerno
Fisciano (SA), Italy
carferrara@unisa.it

Francesco Casillo 
University of Salerno
Fisciano (SA), Italy
fcasillo@unisa.it

Andrea De Lucia 
University of Salerno
Fisciano (SA), Italy
adelucia@unisa.it

ABSTRACT

Machine learning (ML) is increasingly being used as a key component of most software systems, yet serious concerns have been raised about the fairness of ML predictions. Researchers have been proposing novel methods to support the development of fair machine learning solutions. Nonetheless, most of them can only be used in late development stages, e.g., during model training, while there is a lack of methods that may provide practitioners with early fairness analytics enabling the treatment of fairness throughout the development lifecycle. This paper proposes ReFAIR, a novel context-aware requirements engineering framework that allows to classify sensitive features from User Stories. By exploiting natural language processing and word embedding techniques, our framework first identifies both the use case domain and the machine learning task to be performed in the system being developed; afterward, it recommends which are the context-specific sensitive features to be considered during the implementation. We assess the capabilities of ReFAIR by experimenting it against a synthetic dataset—which we built as part of our research—composed of 12,401 User Stories related to 34 application domains. Our findings showcase the high accuracy of ReFAIR, other than highlighting its current limitations.

CCS CONCEPTS

• Software and its engineering → Requirements analysis.

KEYWORDS

Software Fairness; Machine Learning; Requirements Engineering.

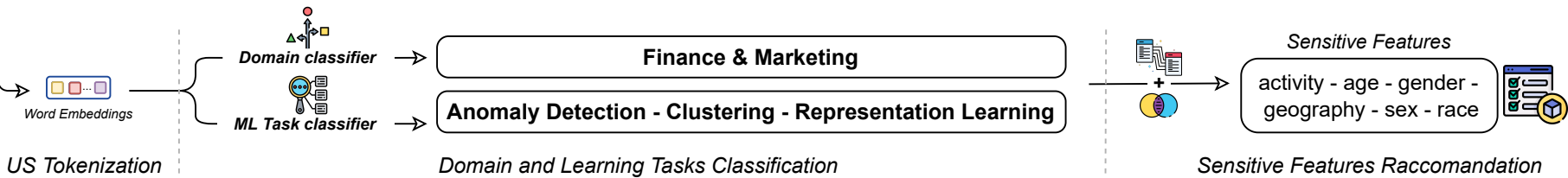
ACM Reference Format:

Carmine Ferrara , Francesco Casillo , Carmine Gravino , Andrea De Lucia , and Fabio Palomba . 2024. ReFAIR: Toward a Context-Aware Recommender for Fairness Requirements Engineering. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*, April 14–20, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3597503.3639185>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ICSE '24, April 14–20, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0217-4/24/04.
<https://doi.org/10.1145/3597503.3639185>

As a **marketer**, I want to use **nearest neighbor search** to identify customers with similar preferences or behaviors, so that I can provide **personalized marketing messages** and improve **customer engagement**.

 Actor  Action  Benefits



1

1 INTRODUCTION

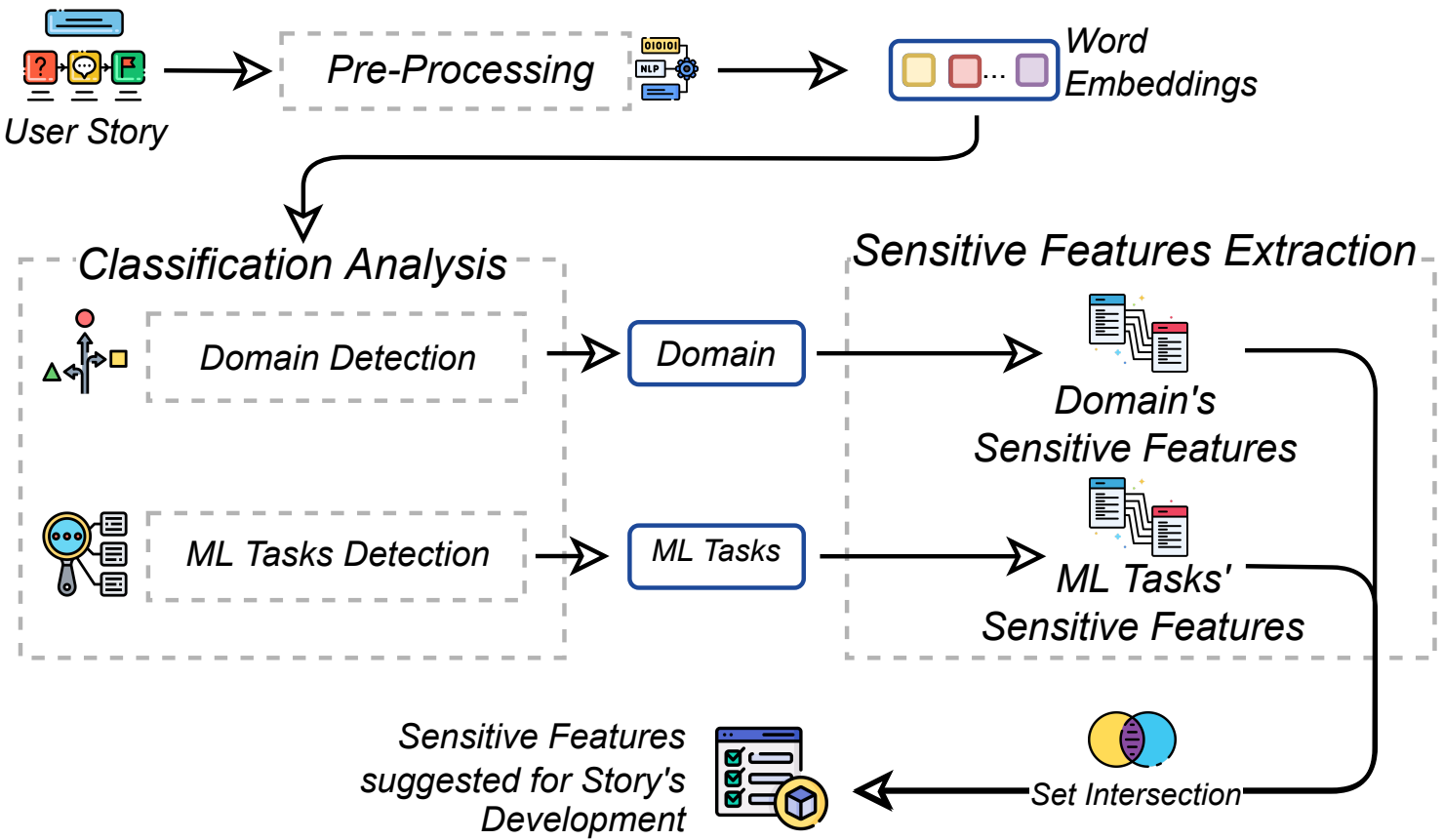
In today's software production, data-driven machine learning (ML) algorithms are more and more employed to support making activities performed by individuals and companies other than to automate repetitive tasks, reducing human load [51]. Successful applications have been showcased in domains [23] like loan management [46], hiring decisions [2], biology [56], and more.

Despite the benefits brought by ML, researchers have reported on the implications that those algorithms may have on ethics and fairness [38]: the reliance on historical data, an ML algorithm to gather biased knowledge about the world, the ruling a phenomenon, which might lead to unfair predictions, recommendations that, in turn, might reiterate discrimination [3]. As such, the definition of methods and tools to reduce risks due to ethical concerns represent a key challenge.

Recently, the *Software Engineering for Artificial Intelligence (SE4AI)* research community has been actively working on this matter, arguing the need for novel engineering processes to ensure fairness throughout the software lifecycle. However, opening this need is complex because of the intrinsic nature of fairness. On the one hand, fairness strictly depends on the application domain and the specific task an ML system is designed for. For example, a feature may be sensitive in one context and not in another. On the other hand, fairness represents a multi-faceted aspect, indeed, multiple definitions targeting various perspectives on fairness have been proposed in literature [60].

At the current stage, most of the existing approaches aim at avoiding discrimination from data. For instance, Chakraborty et al. [14] proposed FAIR-SMOTE, an oversampling algorithm to rebalance training data according to sensitive attributes. On a similar note, other researchers attempted to analyze data preprocessing actions to keep fairness under control, to diversify data to reduce fairness concerns [44], and training data to balance fairness and accuracy [17, 29]. At the same time, automated fairness testing procedures have been developed.

Recognizing these advances, Soremekun et al. [54] point out the need for novel requirements engineering techniques that let practitioners be aware of sensitive features since the inception. Those instruments may have a fundamental impact in practice: being able to provide early recommendations on sensitive features, they may inform the entire ML engineering



Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How?

ARTIFICIAL INTELLIGENCE FOR REQUIREMENTS ENGINEERING INTERMEZZO

Why is AI useful in this case?

Requirements engineering is an intrinsically hard problem, as it mirrors human perception, which is often ambiguous, evolving, and context-dependent.

Most of you conducted a software engineering project: in that context, requirements changed very frequently and you experienced the hardness to elicit a complete set of requirements. At the same time, gathering non-functional requirements is even worst...

Consider the ethical aspects of artificial intelligence. These are naturally context-dependent: for instance, discriminating men and women is wrong in recruitment systems, but may be fine in medical circumstances, as men and women are biologically more or less inclined to certain pathologies.

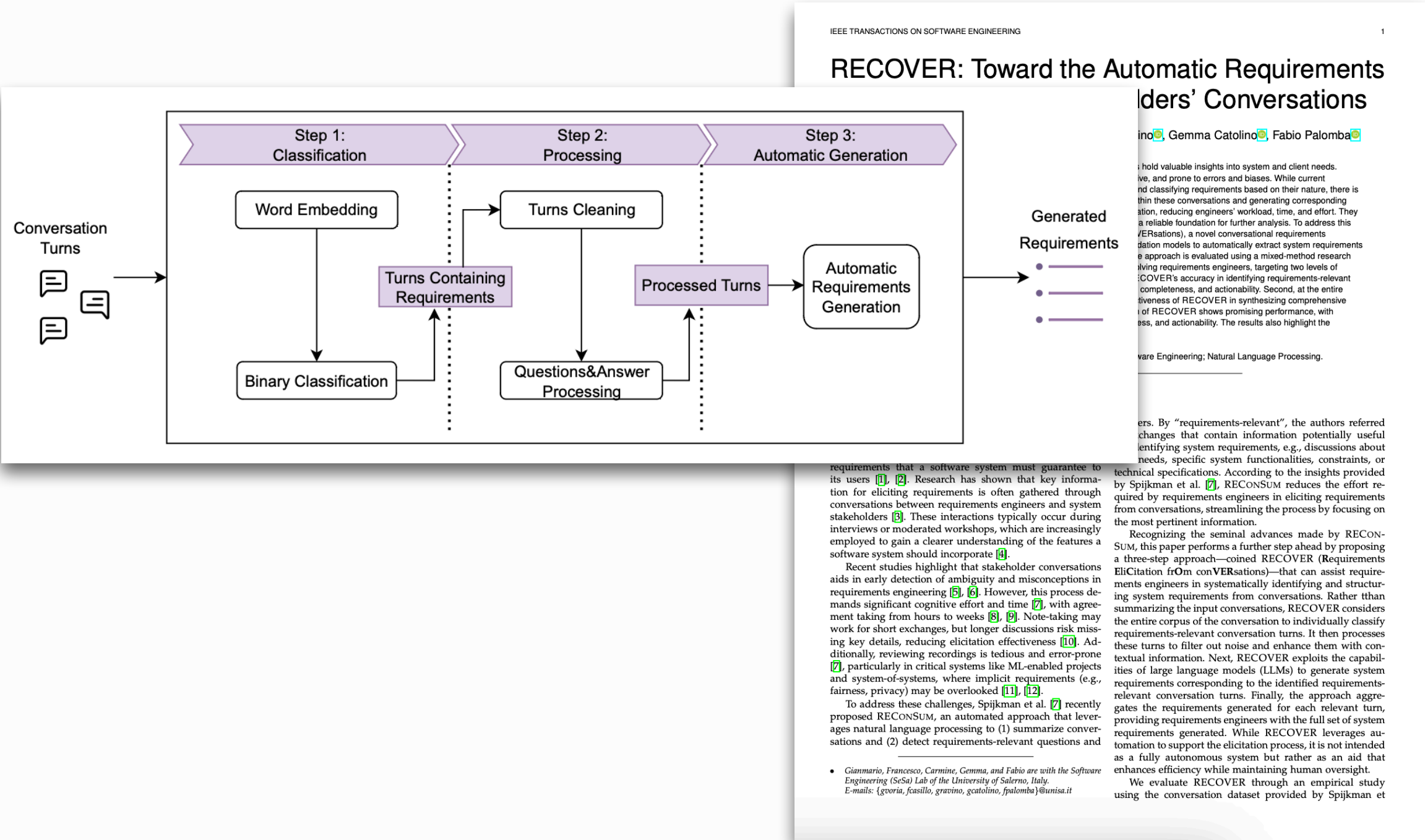
Furthermore, non-functional requirements are often not explicitly discussed by stakeholders and, as such, techniques that may support their elicitation would be helpful.

This is why additional attempts are being made to elicit requirements earlier from stakeholders conversations.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How?

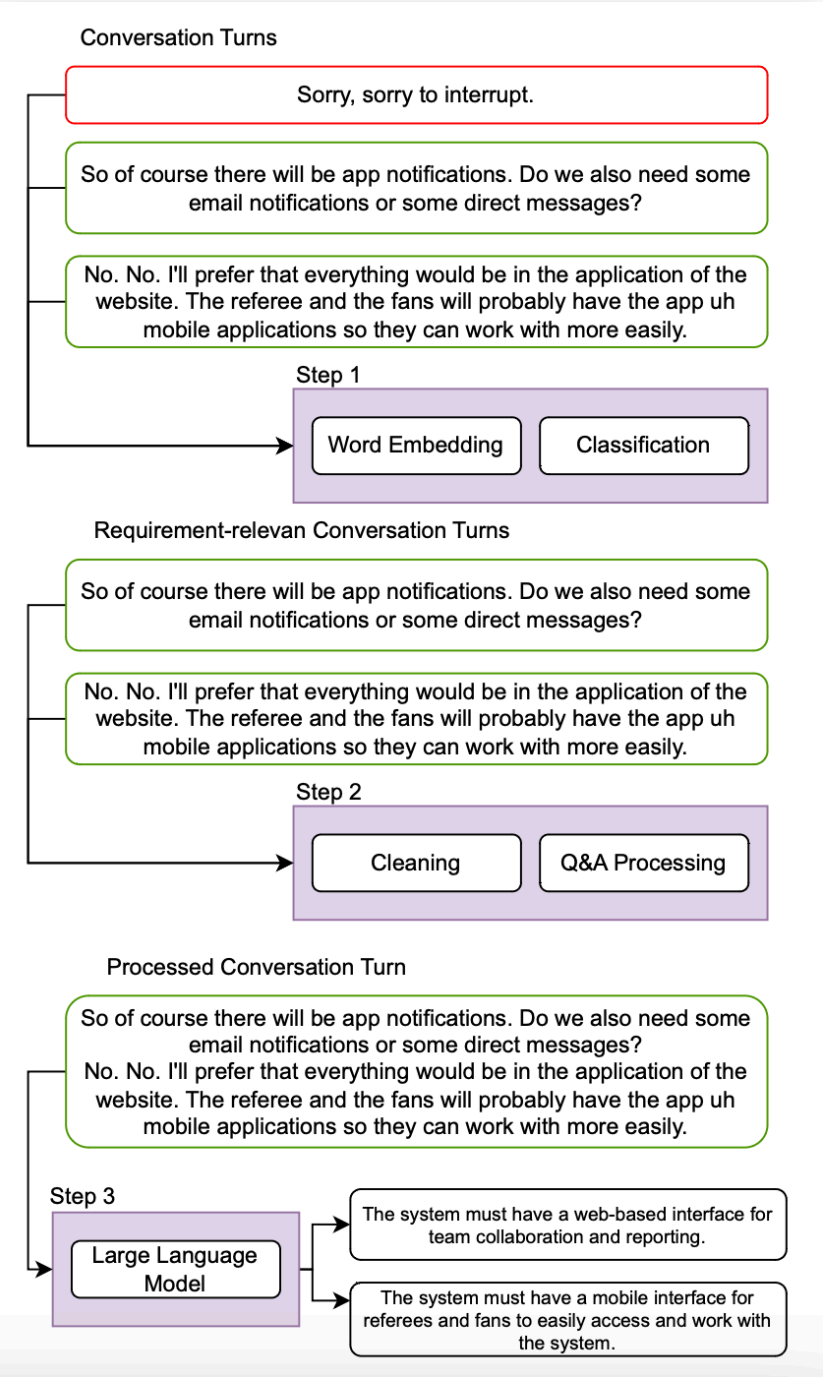
ARTIFICIAL INTELLIGENCE FOR REQUIREMENTS ENGINEERING INTERMEZZO



Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How?

ARTIFICIAL INTELLIGENCE FOR REQUIREMENTS ENGINEERING INTERMEZZO



IEEE TRANSACTIONS ON SOFTWARE ENGINEERING

RECOVER: Toward the Automatic Requirements Elicitation from Stakeholders' Conversations

Gemma Catolino, Fabio Palomba

hold valuable insights into system and client needs. However, current approaches for identifying requirements based on their nature, there is a need for a reliable foundation for further analysis. To address this, RECOVER (Requirements Elicitation from Stakeholders' Conversations), a novel conversational requirements elicitation model to automatically extract system requirements from stakeholders' conversations, is evaluated using a mixed-method research approach involving requirements engineers, targeting two levels of RECOVER's accuracy in identifying requirements-relevant conversation turns: completeness, and actionability. Second, at the entire conversation level, RECOVER shows promising performance, with high precision, recall, and actionability. The results also highlight the

Software Engineering; Natural Language Processing.

requirements that a software system must guarantee to its users [1], [2]. Research has shown that key information for eliciting requirements is often gathered through conversations between requirements engineers and system stakeholders [3]. These interactions typically occur during interviews or moderated workshops, which are increasingly employed to gain a clearer understanding of the features a software system should incorporate [4].

Recent studies highlight that stakeholder conversations aids in early detection of ambiguity and misconceptions in requirements engineering [5], [6]. However, this process demands significant cognitive effort and time [7], with agreement taking from hours to weeks [8], [9]. Note-taking may work for short exchanges, but longer discussions risk missing key details, reducing elicitation effectiveness [10]. Additionally, reviewing recordings is tedious and error-prone [11], particularly in critical systems like ML-enabled projects and system-of-systems, where implicit requirements (e.g., fairness, privacy) may be overlooked [11], [12].

To address these challenges, Spijkman et al. [7] recently proposed RECONSUM, an automated approach that leverages natural language processing to (1) summarize conversations and (2) detect requirements-relevant questions and

ers. By "requirements-relevant", the authors referred to changes that contain information potentially useful for identifying system requirements, e.g., discussions about needs, specific system functionalities, constraints, or technical specifications. According to the insights provided by Spijkman et al. [7], RECONSUM reduces the effort required by requirements engineers in eliciting requirements from conversations, streamlining the process by focusing on the most pertinent information.

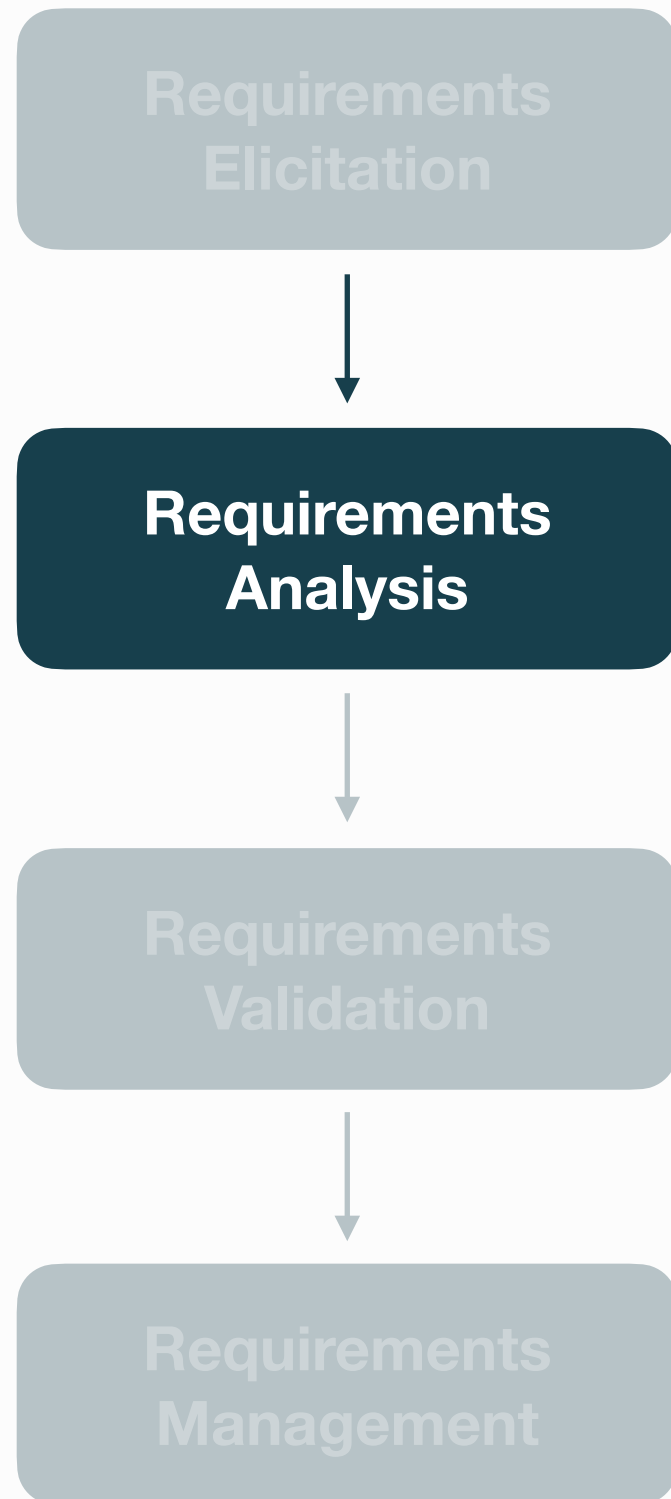
Recognizing the seminal advances made by RECONSUM, this paper performs a further step ahead by proposing a three-step approach—coined RECOVER (Requirements Elicitation from Stakeholders' Conversations)—that can assist requirements engineers in systematically identifying and structuring system requirements from conversations. Rather than summarizing the input conversations, RECOVER considers the entire corpus of the conversation to individually classify requirements-relevant conversation turns. It then processes these turns to filter out noise and enhance them with contextual information. Next, RECOVER exploits the capabilities of large language models (LLMs) to generate system requirements corresponding to the identified requirements-relevant conversation turns. Finally, the approach aggregates the requirements generated for each relevant turn, providing requirements engineers with the full set of system requirements generated. While RECOVER leverages automation to support the elicitation process, it is not intended as a fully autonomous system but rather as an aid that enhances efficiency while maintaining human oversight.

We evaluate RECOVER through an empirical study using the conversation dataset provided by Spijkman et

• Gianmario, Francesco, Carmine, Gemma, and Fabio are with the Software Engineering (SeSa) Lab of the University of Salerno, Italy. E-mails: {ggoria, fcasillo, gravino, gcatolino, fpalomba}@unisa.it

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How?



The objective of the analysis phase is to produce formal software requirements models.

In this stage, additional knowledge might be required as a consequence of the incomplete elicitation of the requirements. The process is to be considered iterative.

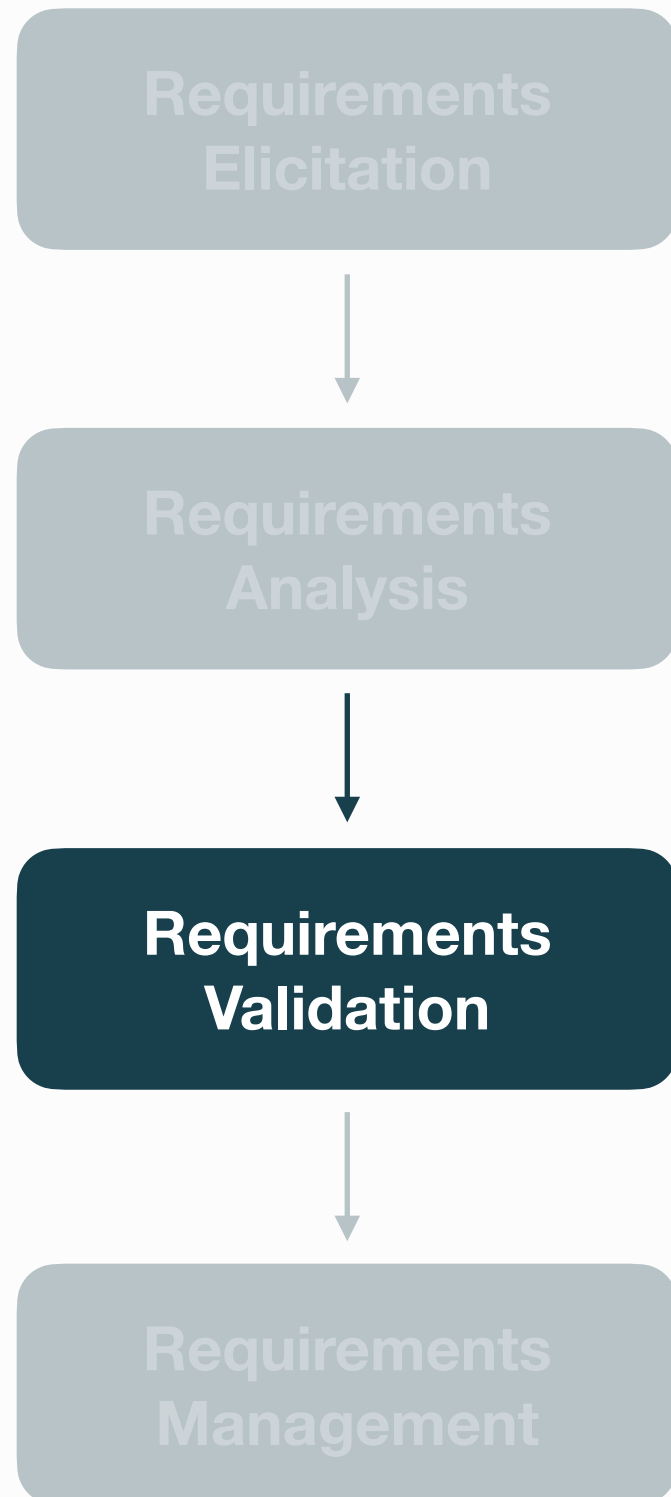
Various models can be used to support requirement analysis:

- (1) *Entity-Relation Diagrams*. Models used to specify the data that should be stored in the system;
- (2) *Data Flow Diagrams*. Models used to specify the dynamic behavior of the data, namely how the data are exchanged via the various components of the system.
- (3) *Dynamic Diagrams*. Models used to specify the dynamic behavior of the system, namely how the various functionalities interact to accomplish complex tasks.

The analysis phase ends with the definition of those diagrams, which bridge the gap between what customers want and what analysts understand.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How?



The objective of the analysis phase is to verify and validate the requirements elicited from the customer.

In particular, we refer to:

(1) *Verification*. The set of tasks that the software correctly implements a specific function;

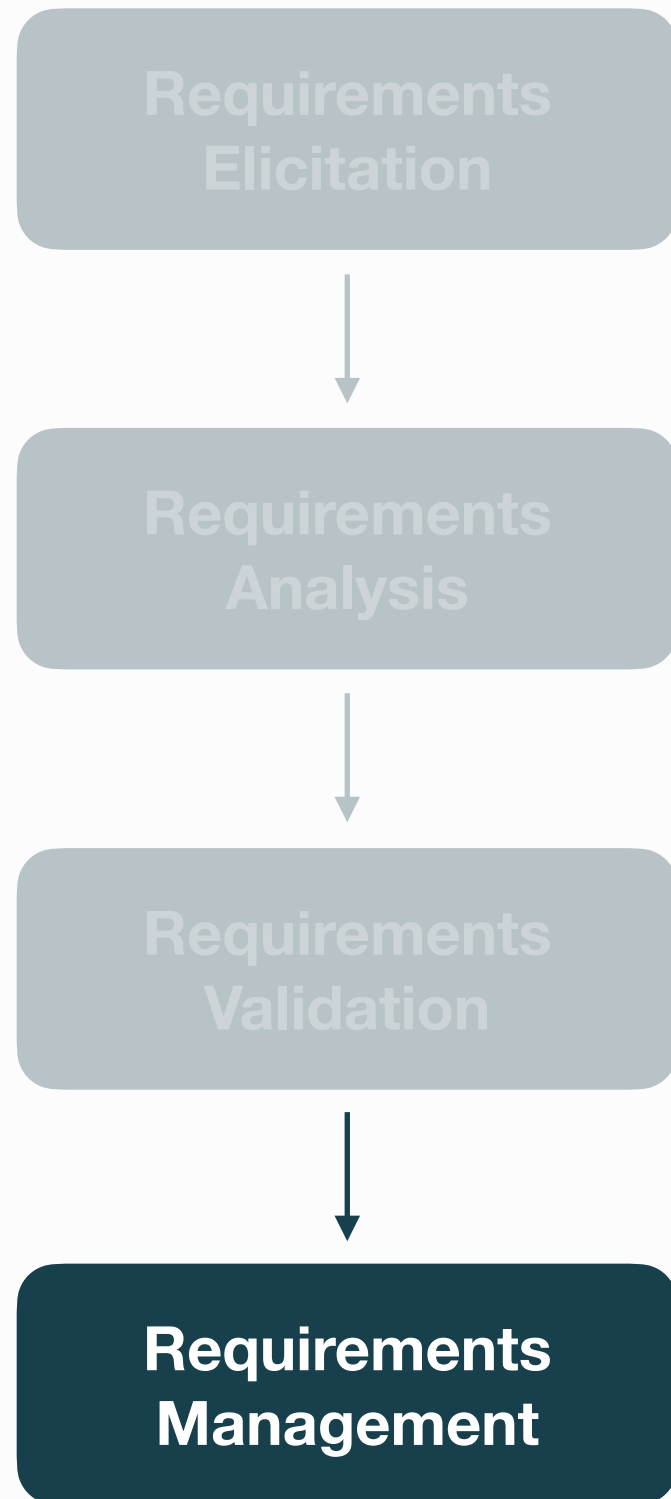
(2) *Validation*. The set of tasks that the software built is traceable to customer requirements.

While verification is an internal process aiming at testing the correct implementation, validation might be considered as an external process that aims at assessing whether all requirements have been considered and, therefore, if customers will be satisfied by the set of functionalities made available.

As any software system, even AI-intensive projects need to be properly tested: nonetheless, testing AI components is not really the same as testing a piece of code. They might be non-deterministic, e.g., how to verify that a machine learning model does what it is supposed to do?

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How?



The objective of this phase is to monitor the evolution of the requirements and ensure that all are implemented.

With respect to the other phases, the management is an activity that lasts for the entire life cycle of the development.

Typically, the management starts with the definition of documents collecting requirements and models —> you have known this as the Requirements Analysis Document (RAD), but alternative solutions are possible, like the collection of user stories in agile development.

Right after collecting requirements, the management should include the definition of the verification & validation procedures —> you have known this as the Test Plan (TP).

(Yes, test plans should be done right after requirements analysis!)

Afterwards, managers and developers keep monitoring the requirements by means of traceability matrices and other typical software engineering mechanisms (e.g., code reuse, definition of design goals, etc.).

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How Should Requirements Be?

The overall goal of the requirements engineering process is to output a set of requirements that are in line with multiple qualities.

Completeness

The set of requirements elicited must be complete, namely all the customer's expectations should be mapped onto (non-)functional requirements.

Correctness

The set of requirements elicited must correctly map the customer's expectations, namely they should properly report what the customer asks.

Feasibility

The set of requirements elicited must be actually implementable within the system —> this is particularly relevant when discussing AI components.

Necessity

The set of requirements elicited must be actually needed within the system, namely the system should contain the minimum functionalities possible. Why?

Prioritization

The set of requirements elicited must be prioritized so that the release engineering processes can be designed and implemented.

Unambiguity

The set of requirements elicited must be clear to avoid multiple interpretations that might lead to misunderstandings and/or errors.

Verifiability

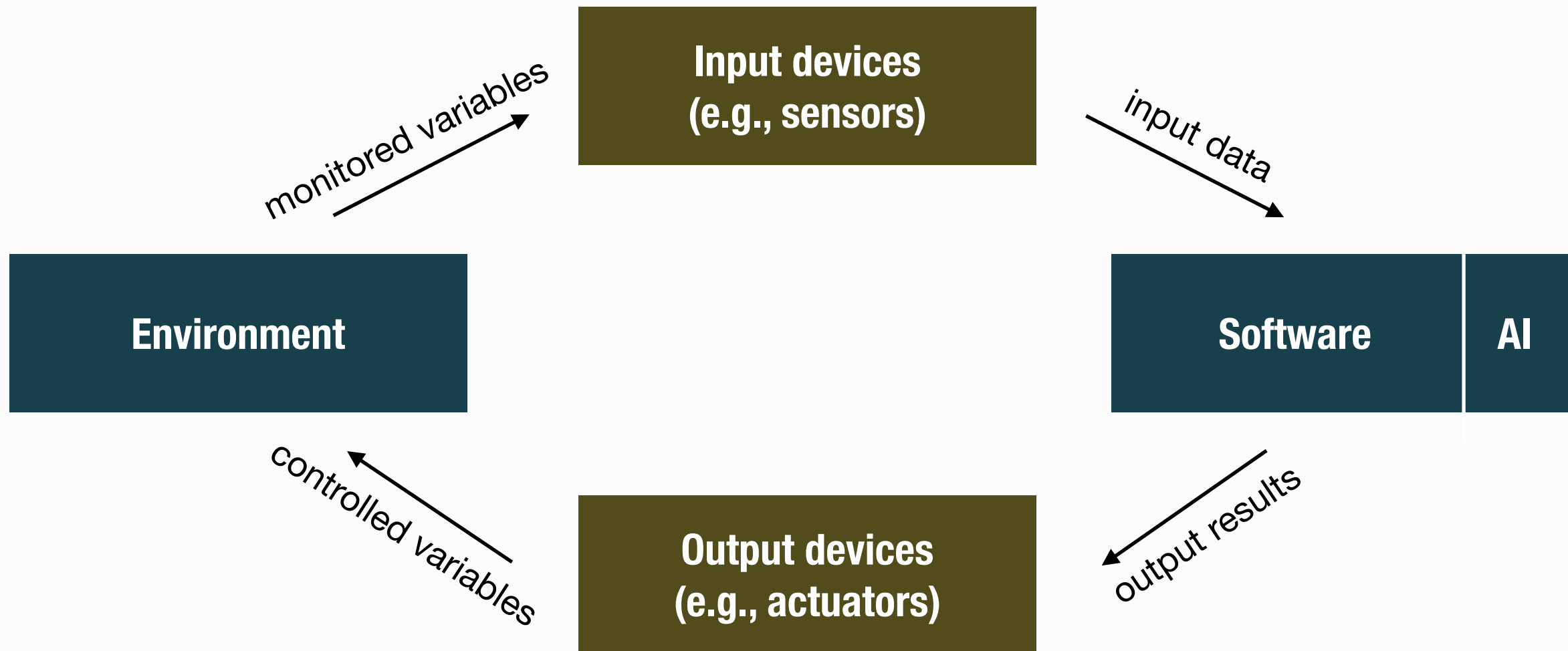
The set of requirements elicited must be expressed so that they can be verified —> *"You cannot control what you cannot measure"* (Tom De Marco).

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Implications for AI-intensive systems

What does this mean for AI-intensive systems?

No software lives in vacuum; every system is deployed as part of the world.



A requirement describes a desired state of the world (i.e., environment).

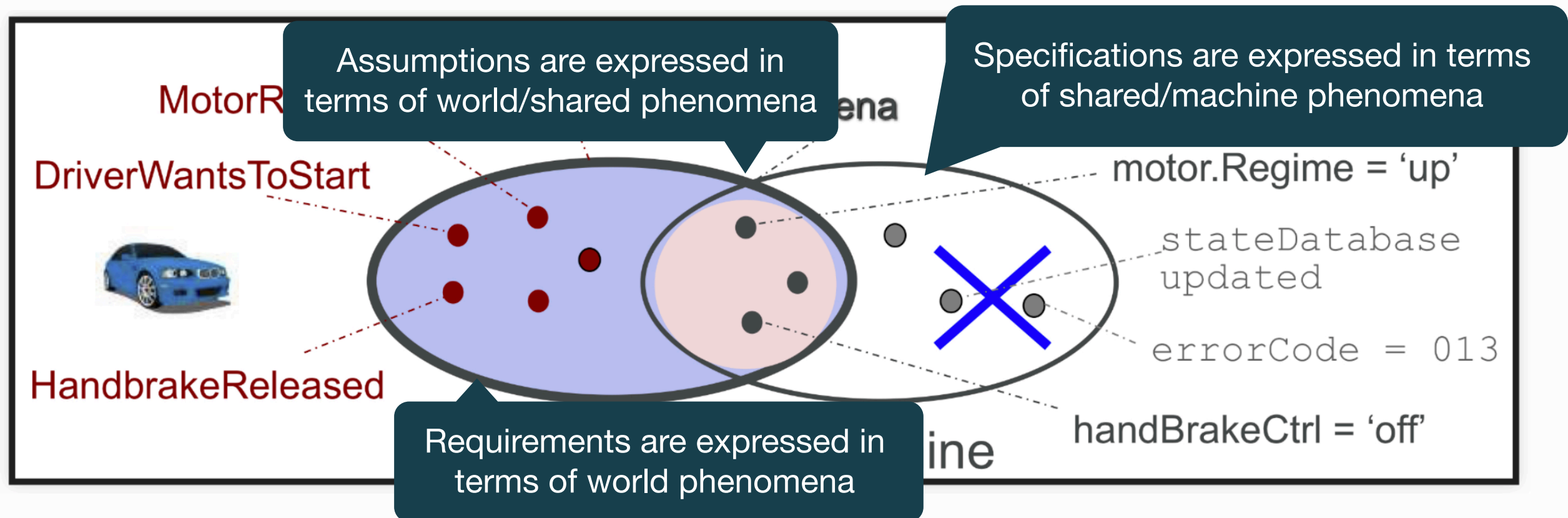
Software is created to manipulate the environment into this desired state.

Yet, this view is still too limited... Environment and software are way (way way) more interconnected in reality!

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Implications for AI-intensive systems

Let's consider the example of a self-driving car.



Shared phenomena: Interface between the world and the machine (i.e., the software).

Requirements model stakeholder's needs and are expressed in natural language in order to be understood by all interested parts (e.g., "*the driver wants to start*").

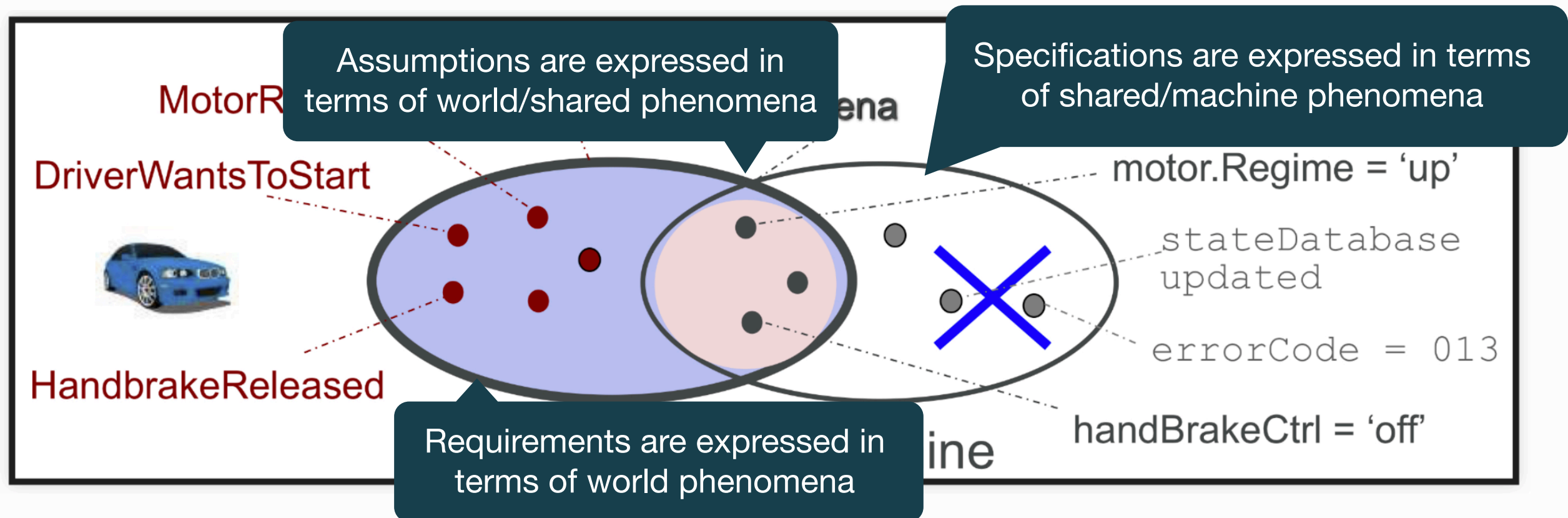
Assumptions are needed to abstract the surrounding world and must necessarily consider the way customers interact with the system (e.g., `motor.Regime = 'up'`).

Requirement specifications map how the system reacts to requirements and assumptions (e.g., `stateDatabase updated`).

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Implications for AI-intensive systems

Let's consider the example of a self-driving car.



software).

uage in order

sarily

= 'up').

d

What could go wrong?

- (1) Missing/Incorrect environmental assumptions;
- (2) Wrong specifications;
- (3) Inconsistency in assumptions and specifications;
- (4) Inconsistency in requirements.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Implications for AI-intensive systems

Lufthansa 2904 Runway Crash



Reverse Thrust: Decelerates plane during landing;

What was required: reverse thrust enabled if and only if plane on the ground;

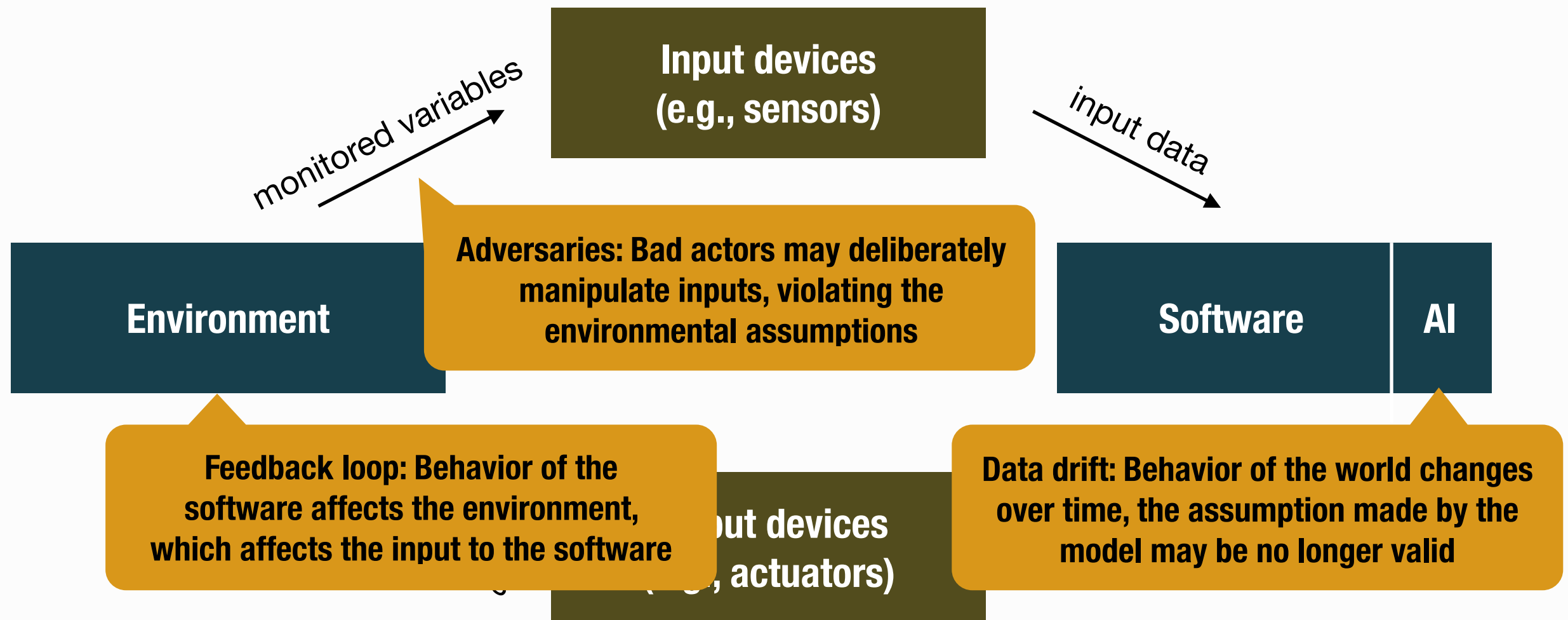
What was implemented: reverse thrust enabled if and only if wheel turning;

But: Runway wet + wind, wheels did not turn, pilot overridden by software —> crash!

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Implications for AI-intensive systems

A few additional considerations on how artificial intelligence is impacted by the requirements engineering process:



These are the reasons why, nowadays, having an ML-Ops pipeline becomes crucial. It allows software engineers and data scientists to continuously measure environment, assumptions, and quality of the AI models, so that preventive actions might be done.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Implications for AI-intensive systems

A few additional considerations on how artificial intelligence is impacted by the requirements engineering process:

Indeed, (non-)functional requirements have their own lifecycle and should be considered at different development times.

Perhaps more importantly, feedback loops, adversarial attacks, and evolution may lead to the erosion of non-functional attributes, i.e., monitoring and testing are key!

NFR	Cluster	System Element the NFR Can be Defined Over				
		Train. Data	Algo.	Model	Results	Whole System
Completeness	1	✓	✗	✓	✗	✓
Correctness	1	✓	✓	✓	✓	✓
Fault Tolerance	1	✗	✓	✓	✗	✓
Integrity	1	✓	✓	✓	✓	✓
Repeatability	1	✗	✗	✗	✓	✓
Explainability	2	✗	✓	✓	✓	✓
Transparency	2	✗	✓	✓	✓	✓
Ethics	3	✓	✓	✓	✓	✓
Fairness	3	✓	✓	✓	✓	✓
Adaptability	5	✓	✓	✓	✓	✓
Domain Adaptation	5	✓	✓	✓	✓	✓
Flexibility	5	✗	✓	✓	✗	✓
Interoperability	5	✗	✓	✓	✗	✓
Portability	5	✓	✓	✓	✗	✓
Reusability	5	✓	✓	✓	✗	✓
Maintainability	7	✓	✓	✓	✗	✓
Testability	7	✗	✓	✓	✓	✓
Traceability	7	✓	✓	✓	✓	✓
Usability	7	✗	✓	✓	✓	✓

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How to Deal with those Implications

Spoiler: Most of the aspects treated in the course are still a work-in-progress. Big companies, EU commission, and other institutions are investing a lot of money to support research around the requirements of AI-intensive systems. If you are interested, just drop me a message. We have ideas and money!

As a matter of fact, the artificial intelligence community has explicitly required the intervention of software engineers to mitigate the many risks of using AI.

Up to now, the most established method to deal with the complexity of modeling AI components while keeping non-functional requirements into account is called *trade-off analysis* (some of you might remember the concept of trade-off during the system and object design of your software engineering projects).

Here the concept is similar, but its application is way harder!

Essentially, the core idea is to find a compromise between the many aspects that can influence the development of effective AI applications.

The problem is that there are A LOT of aspects to consider. Some of you - at least those who attended Fundamentals of Artificial Intelligence - might think at solutions that can deal with cases where multiple aspects should be balanced... any ideas?

—> Multi-objective optimization!

NB: We do not have time to talk about multi-objective optimization but, if you are interested, you will find something in the additional material!

Requirements Engineering: How to Deal with those Implications

Netflix Recommendations: Beyond the 5 stars (Part 1)

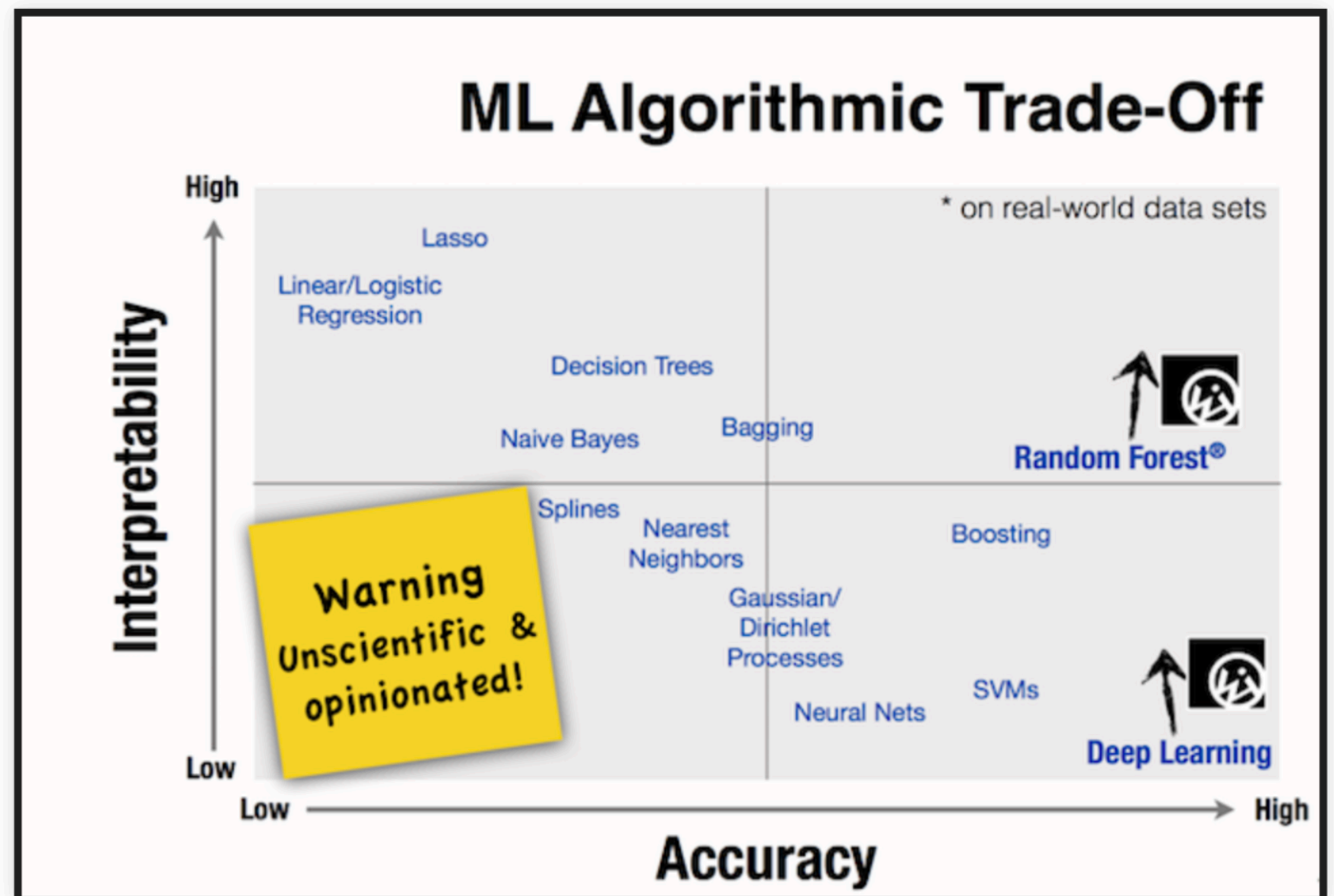


Netflix Technology Blog

Follow

Apr 6, 2012 · 9 min read

<https://netflixtechblog.com/netflix>



<https://www.oreilly.com/library/view/strata-conference-santa/9781491900321/oreillyvideos1977272.html>

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How to Deal with those Implications

Netflix Recommendations:

Reminder

"You cannot control what you cannot measure"

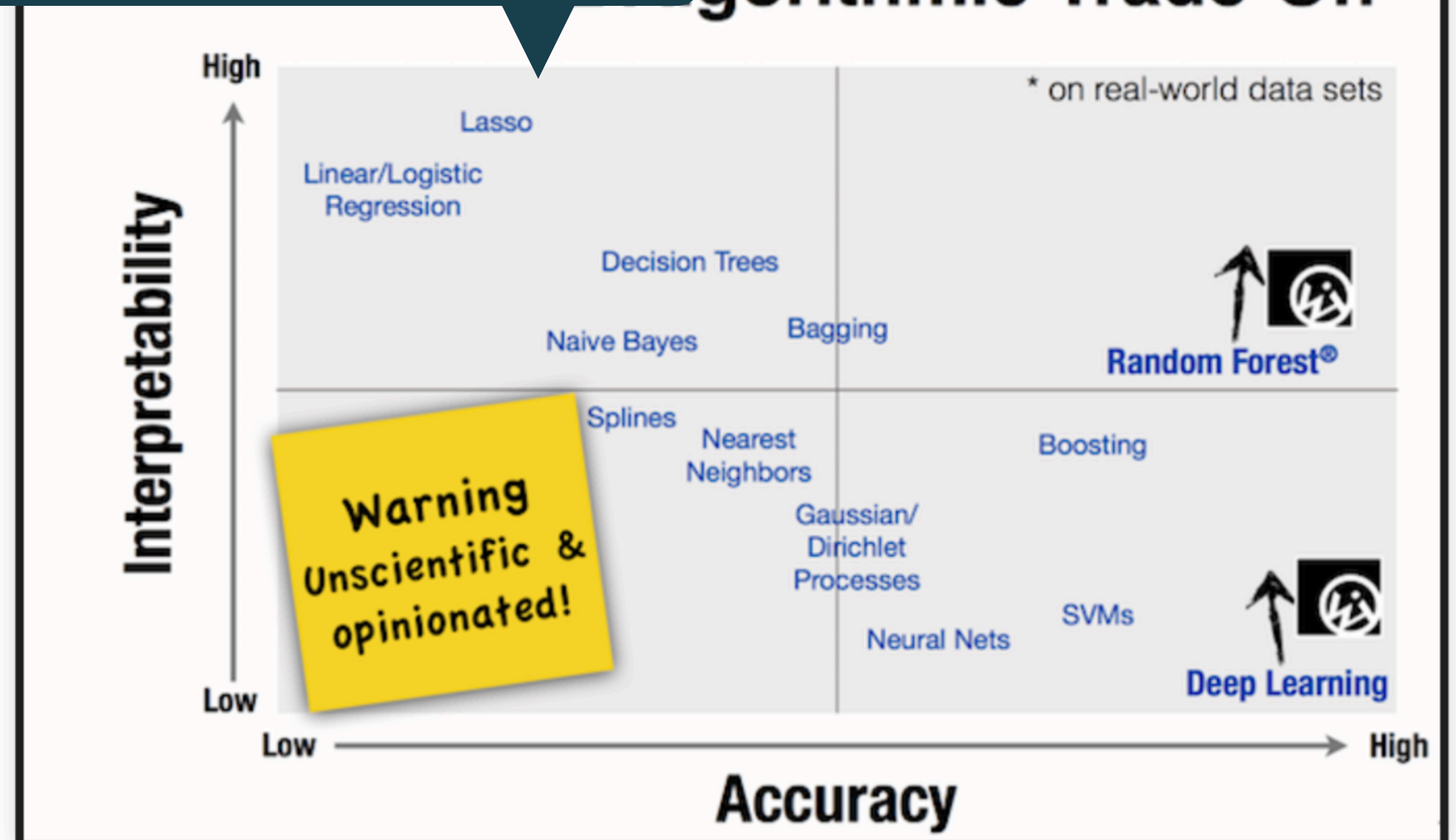
The reason why we obsessed you with having quantifiable non-functional requirements at the SE course is because you can use them to take informed business decisions!

<https://netflixtechblog.com/netflix>

Multi-objective optimization allows to generalize the search to an arbitrary number of constraints!

To enable optimization, you need software analytics instruments: some of them are becoming available, but there is a long way to go...

Algorithmic Trade-Off



<https://www.oreilly.com/library/view/strata-conference-santa/9781491900321/oreillyvideos1977272.html>

Software Engineering for Artificial Intelligence: Requirements Engineering

Before leaving... What Should You Remember?

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Why?

During the Bachelor course on software engineering, you had to study the requirements elicitation and analysis processes - I guess that many of you vividly remember that part of the course as the most challenging or, at least, as the activity that most frequently led to changes.

Unfortunately, that was “normal”: the requirements engineering process is the hardest to complete, as requirements change over time and stakeholders might change their mind during the development process.

Artificial intelligence-enabled systems are even more complex to analyze from the perspective of requirements engineering: you do not only have to model the surrounding system, but also the properties of the artificial intelligence agent.

Requirement. A statement, typically developed in natural language, that provides indications on a system property and its operational constraints.

Requirements Engineering. The process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed.

In essence, the process aims at investigating, identifying, communicating, and documenting the properties of a system.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: A New Set of Non-Functional Requirements?

Some non-functional requirements are indeed different, others may assume a different meaning or have different implications.

Quality The ability of an AI component to be sound, stable, and reliable.
Examples: Accuracy, retrainability, testability, usability, interpretability, flexibility, explainability.

Infrastructure The ability to build, run, and manage AI components.
Examples: Scalability, performance, fault tolerance, efficiency.

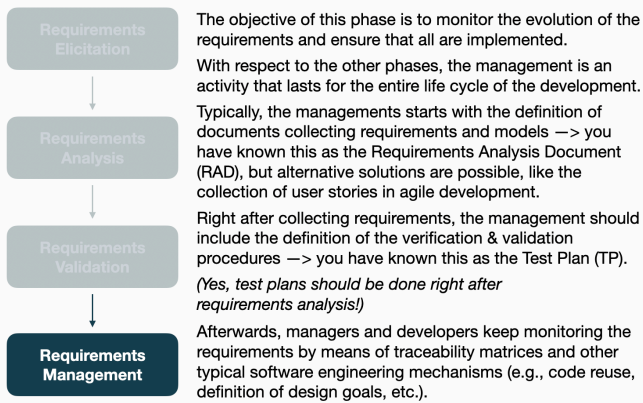
Ethics The ability of an AI component to act respecting laws and ethics.
Examples: Fairness, transparency, trust, bias.

Safety The ability of an AI component not to cause harm to externals.
Examples: Privacy, data access.

Security The ability of an AI component to secure the data it manages.
Examples: Integrity, data issues.

Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How?



Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How Should They Be?

The overall goal of the requirements engineering process is to output a set of requirements that are in line with multiple qualities.

Completeness The set of requirements elicited must be complete, namely all the customer's expectations should be map onto (non-)functional requirements.

Correctness The set of requirements elicited must correctly map the customer's expectations, namely they should properly report what the customer asks.

Feasibility The set of requirements elicited must be actually implementable within the system —> this is particularly relevant when discussing of AI components.

Necessity The set of requirements elicited must be actually needed within the system, namely the system should contain the minimum functionalities possible. Why?

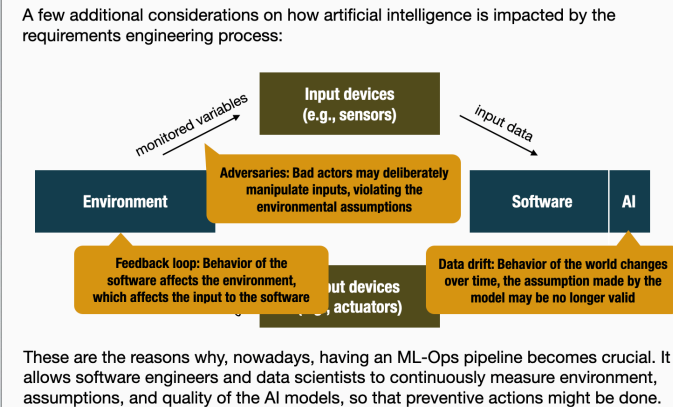
Prioritization The set of requirements elicited must be prioritized so that the release engineering processes can be designed and implemented.

Unambiguity The set of requirements elicited must be clear to avoid multiple interpretations that might lead to misunderstandings and/or errors.

Verifiability The set of requirements elicited must be expressed so that they can be verified —> “You cannot control what you cannot measure” (Tom De Marco).

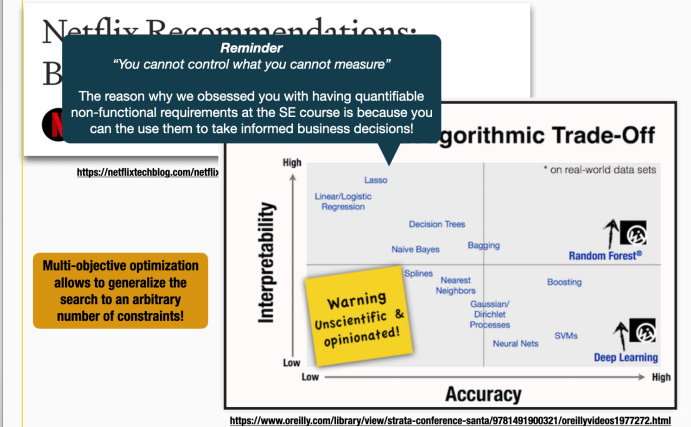
Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: Implications for AI-intensive systems



Software Engineering for Artificial Intelligence: Requirements Engineering

Requirements Engineering: How to Deal with those Implications



Software Engineering for Artificial Intelligence

Requirements Engineering for AI-based Systems

